

PCAP2STORY: Fault Detection and Human-Readable Explanation for EVPN Control Plane Traffic

Jayashree Kempnanahalli Shivakumar



Master of Science
Data Science
School of Informatics
University of Edinburgh
2026

Abstract

EVPN (Ethernet Virtual Private Network) control plane faults are slow and labour-intensive to diagnose manually, and to our knowledge, no published detection or explanation method targets them specifically. Existing work on Border Gateway Protocol (BGP) anomaly detection targets a different problem: disputes between separate operators, such as route hijacking or route leaking, not faults confined entirely within one operator’s own EVPN network. The closest systems that operate on raw packet captures, PCAPVision and LLMcap, target failures in telecom phone-call setup and signaling, not EVPN, and neither grounds its output in a written specification.

PCAP2STORY is a three-stage pipeline: extracting events from each capture and merging them into one timeline, detection, and explanation. Detection uses Request for Comments (RFC) grounded rule modules, one per fault type, each checking a merged event timeline against a trigger condition written by hand from the relevant RFC clause. A precedence step resolves cases where more than one rule fires for the same event. Explanation is generated by a language model, grounded in retrieved RFC text, the network’s topology, and suggests a next step for the engineer to check.

The dataset combines 369 real packet captures, from a small emulated test network running FRR (Free Range Routing) and 74 synthetic captures for scenarios the testbed could not reliably produce, the first labelled dataset for EVPN control plane faults. Detection matched every ground-truth label in the corpus with zero false positives. Explanation quality was evaluated through five separate methods, since no single correct explanation exists to check against. Each surfaced a real defect: a search query that consistently pulled up the wrong RFC section, a scope error caught independently when two different language models were used to check the explanation, and a separate retrieval gap that made two compatible RFC sections look like a contradiction. Three correction methods were then compared. The method used in the final pipeline was not the one that resolved the most files. It was chosen because it refused to guess when the evidence it was given was incomplete, rather than producing a confident wrong answer.

These results support RFC-grounded detection paired with an explanation whose claims can be checked against that same specification. Among the systems reviewed, none detects EVPN faults from raw packet captures and grounds its explanation in a written specification the way PCAP2STORY does.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Jayashree Kempanahalli Shivakumar)

Acknowledgements

This dissertation would not have taken the shape it did without the people who supported and inspired me.

I thank the School of Informatics, University of Edinburgh, for the teaching and opportunities that shaped this degree.

I am grateful to my supervisor Mahesh Marina, and to Tianxin Wang, for their technical guidance throughout this project and for holding a high standard that made this a much stronger piece of work.

I owe Scott Leishman and David Stapleton at Ciena for their constant support throughout this project. Beyond their domain expertise, I am grateful to them for trusting a student with a project this open-ended, one with real breadth across machine learning and large language models, and for giving me the room to learn that space from scratch rather than handing me a narrow, pre-solved problem.

Finally, I thank my parents and my sister, whose support made me pursue this Master's in the first place and their daily encouragement carried me through the harder stretches of it. Their belief in me mattered more than they probably realise, and this work is as much a product of their support as it is of my own effort.

Use of Generative AI. Generative AI tools (Claude, Anthropic, Gemini, and GPT) were used to analyse and understand codebases, debug testbed and pipeline code, clarify networking terminology and protocol behaviour, and for formatting and clarification of written text. All architectural decisions, experimental design, analysis, and conclusions are my own.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Hypothesis	3
1.3	Contributions	3
1.4	Thesis Structure	4
2	Background	5
2.1	BGP and EVPN Fundamentals	5
2.2	Fault Types in EVPN Deployments	6
2.3	Related Work	8
2.3.1	Research Gap	11
3	Dataset Generation	12
3.1	Overview	12
3.2	Real Captures	13
3.3	Synthetic Captures	15
3.4	Scope Boundaries	15
4	System Design and Implementation	16
4.1	System Overview	16
4.2	Stage 1: Event Extraction and Fusion	17
4.2.1	Event Extraction	17
4.2.2	Fusion	18
4.3	Stage 2: Detection	19
4.3.1	Rule Modules	19
4.3.2	Precedence	21
4.4	Stage 3: Explanation	23
4.4.1	Retrieval	23

4.4.2	Generation	24
4.4.3	Correction	25
5	Evaluation	27
5.1	Evaluation Methodology	27
5.2	Detection Accuracy	27
5.3	Explanation Quality	28
5.3.1	Ablation Study	28
5.3.2	Groundedness Check	29
5.3.3	Atomic Claim Decomposition	30
5.3.4	Self-Consistency	31
5.3.5	LLM as Judge	32
5.3.6	Correction Outcomes	34
5.3.7	Discriminability Across Trigger Mechanisms	35
6	Conclusion	37
6.1	Summary	37
6.2	Limitations	38
6.3	Future Work	39
	Bibliography	41
A	Dataset Generation Details	47
A.1	Real Captures	47
A.1.1	Trigger Mechanisms	47
A.1.2	Scenario Counts	49
A.2	Synthetic Captures	49
A.2.1	Generation Method	49
A.2.2	Scenario Counts	51
A.3	Infrastructure Challenges	51
B	Rule Module Details	53
C	Explanation Examples	57

Glossary

3GPP 3rd Generation Partnership Project - The standards body that publishes telecom specifications, playing the same role for cellular networks that RFCs play for Internet protocols.

802.11 The IEEE standard defining Wi-Fi wireless LAN protocols.

Access Circuit The physical connection between a PE and a CE. If it goes up or down, this can trigger Designated Forwarder re-election.

ADVERTISEMENT A BGP message announcing that the sender can now reach a given destination, so the peer should route through it.

BERTScore A metric built on BERT (Bidirectional Encoder Representations from Transformers) that compares two texts using contextual embeddings rather than exact word matches.

BFD Bidirectional Forwarding Detection - A fast heartbeat between two routers used to detect a lost connection within milliseconds, faster than BGP's own keepalive mechanism.

BGP Border Gateway Protocol - The core protocol that lets separate, independently run networks tell each other which destinations they can reach and through which path. Each network then chooses which path to use based on its own routing policy.

CE Customer Edge - The networking equipment, such as a router or switch, located at the customer's premises. It manages the local network traffic and serves as the direct connection point to the service provider's network.

Cease A BGP notification sent when a session is closed on purpose rather than through failure. It carries a code stating the specific reason, such as a planned shutdown.

containerlab A tool that deploys a defined network topology as a set of connected containers, based on a single topology definition file.

Daemon A background process that runs continuously without needing direct user interaction. FRR's routing software runs as a set of daemons.

DeBERTa-MNLI Decoding-enhanced BERT with disentangled attention (DeBERTa), fine-tuned on Multi-Genre Natural Language Inference (MNLI) - A language model used to check whether one piece of text logically follows from another.

DF Designated Forwarder - The single PE responsible for forwarding traffic onto a shared Ethernet Segment at any given time, chosen through election among the PEs sharing it.

Diagnostic code A field in a BFD message stating the reason for a session state change, such as a heartbeat timeout.

Docker CE Docker Community Edition - A command-line container runtime with no graphical interface, running as a background service.

Docker Desktop A GUI-based application for running Docker on Windows or Mac, bundling the container runtime together with a management interface and a virtualization layer. On Windows, it uses WSL2 as the backend for running Linux containers.

ESDF Ethernet Segment Designated Forwarder - The same role as Designated Forwarder (DF), used specifically in the context of the ESDF Toggle fault type.

ESI Ethernet Segment Identifier - A shared identifier used when a single Customer Edge (CE) connects to more than one Provider Edge (PE) for redundancy. It tells the network that these separate PE connections all lead to the same CE, so they are treated as one logical connection rather than unrelated paths.

Ethernet Segment A shared physical link connecting two or more PEs to the same CE, identified by an ESI.

Ethernet Tag A field in EVPN routes identifying which broadcast domain a route belongs to. Used to distinguish per-EVI signals from per-segment ones during Designated Forwarder election.

EVI **EVPN Instance** - A separate EVPN service running on the same set of PEs, keeping one customer's routes and traffic distinct from another's.

EVPN A networking standard that uses BGP to connect physically separate locations, letting devices on each site see each other as if they were on the same local network.

Extended Community A BGP route attribute used to attach extra information to a route beyond basic reachability, such as Designated Forwarder election state or a MAC mobility sequence number.

FRR Free Range Routing - An open-source software implementation of routing protocols including BGP, OSPF, and BFD.

GPT Generative Pre-trained Transformer - The model architecture behind widely known tools such as ChatGPT, trained to generate text one word at a time based on the preceding text.

Hold Timer A configured countdown that resets each time a keepalive arrives. If it reaches zero before the next one arrives, the session is considered down.

Hyper-V Microsoft's virtualization platform, used by WSL2 to run its Linux virtual machine on Windows.

IP prefix A block of IP addresses defined by a starting address and a length, written together to specify how many addresses the block covers.

JSON JavaScript Object Notation - A text format that stores data as key-value pairs, which can nest to form a tree-like structure.

Keepalive A message sent at configured intervals between two BGP peers to show the session is still active.

Knowledge Graph A structured representation of entities and their relationships, used to ground a language model's reasoning in verifiable facts rather than free generation.

LLM Large Language Model - A machine learning model trained on large amounts of text, able to generate and reason about language.

MAC address Media Access Control address - A hardware identifier assigned to a network interface, tracked by EVPN so traffic can be forwarded to the correct PE.

MAC mobility sequence number A number carried in the MAC Mobility Extended Community. It increases with each move of the same MAC address, distinguishing a genuine repeated move from a stale or duplicate advertisement.

MAC-VRF A separate routing and forwarding table on a PE, keeping one EVPN service's MAC addresses and traffic distinct from another's. A PE can host several MAC-VRFs at once, one per EVPN service.

MTU Maximum Transmission Unit - The largest amount of data a single packet can carry over a link before it must be split into smaller pieces.

Multihoming A CE connected to more than one PE at the same time, so the connection keeps working if one PE fails.

NOTIFICATION A BGP message sent when a session is being closed, carrying a code that identifies the specific reason.

OPEN The first BGP message sent when establishing a session, used to negotiate session parameters before the connection is confirmed.

OSPF Open Shortest Path First - A routing protocol that runs inside a single network to help routers find paths to each other, separate from BGP which handles routing between networks.

PCAP Packet Capture - A saved, byte-level record of every message seen at the point in the network where capture is running.

PE Provider Edge - A routing device at the boundary of a service provider's network. It serves as the gateway that connects a customer's local sites to the provider's main core network.

RAG Retrieval-Augmented Generation - A method where a language model retrieves relevant reference text first. It then generates its output based on what it found, instead of relying only on what it already knows.

RD Route Distinguisher - A value added to a route so that identical-looking routes from different PEs remain distinguishable from one another.

RFC Request for Comments - A published document from the Internet Engineering Task Force that defines how a protocol should work.

RIPE RIS Réseaux IP Européens Routing Information Service - A public BGP monitoring infrastructure similar in purpose to RouteViews. It collects route data from many locations across the internet.

ROUTE-REFRESH A BGP message asking a peer to resend its routes, without any change to reachability itself.

RouteViews A public BGP monitoring project that collects and publishes route traffic from many networks for research use.

RR Route Reflector - A centralized routing device that acts as an information hub. Instead of every router needing a direct connection to every other router to share network paths, they all connect to the Route Reflector, which distributes the information to the entire network.

RT Route Target - value attached to a route that decides which PEs will accept and use it.

TCP Transmission Control Protocol - A software-level protocol that manages a reliable exchange of data between two devices. BGP sessions run on top of it.

Topology The arrangement of nodes and links in a network, and how they connect to each other.

Type-1 route An EVPN route type advertised per Ethernet Segment or per EVI. It supports fast failure detection, traffic redirection among the PEs sharing a segment, and Designated Forwarder election.

Type-2 route An EVPN route type that advertises a MAC address a PE has learned, optionally including its IP address.

Type-3 route An EVPN route type that announces a PE's membership in a given EVPN Instance, used to deliver broadcast and multicast traffic to the right PEs.

Type-4 route An EVPN route type that announces a PE's presence on an Ethernet Segment and carries the Designated Forwarder election state.

Type-5 route An EVPN route type that advertises an IP prefix rather than a specific MAC address.

UPDATE The BGP message type that carries route ADVERTISEMENT and WITHDRAWAL information between peers.

Vantage point A location in the network where packet capture runs, giving visibility into the traffic passing through it.

VLAN Virtual Local Area Network - A logical grouping that segments traffic on a shared network.

VM Virtual Machine - An emulated computer running inside a host system, isolated from it.

Wireshark A widely used tool for opening and manually inspecting packet capture files.

WITHDRAWAL A BGP message signaling that a previously reachable destination should now be removed from the peer's routing table.

WSL2 Windows Subsystem for Linux 2 - A compatibility layer letting a genuine Linux kernel run alongside Windows rather than emulating it.

zebra The core routing manager daemon within FRR. It interfaces with the kernel's routing tables and handles low-level functions such as interface and Ethernet Segment state, distinct from protocol-specific daemons.

Chapter 1

Introduction

1.1 Motivation

Consider an email crossing the internet. It typically passes through several independently operated networks before reaching its destination. Each network along the way needs to know which of its neighbors can carry the message closer to that destination. Border Gateway Protocol (BGP) provides this: a way for independently run networks to tell each other which destinations they can reach, and through which path. Network providers also use BGP for a narrower, related job. Their customers, businesses, data centers, mobile operators, often need several physically separate sites to communicate as if they were on one shared network. Ethernet Virtual Private Network (EVPN) is a way of using BGP to deliver exactly this kind of service, connecting separate sites over infrastructure the provider shares across many customers.

BGP and EVPN carry two different kinds of traffic. The actual data itself travels across what's called the data plane. The traffic that manages how that data moves is the control plane. As the name suggests, it controls the connection, deciding which routers can see each other and who forwards traffic to whom. The control plane can fail for simple reasons, an operator entering the wrong configuration value, a cable failing, or a piece of software crashing. When this happens in the control plane, the data plane suffers even though nothing is wrong with the data itself. The connection carrying it has broken. This makes control plane faults high stakes for a provider. The fault itself may be small, but the consequence is lost customer traffic.

Control plane failures incur significant operational and financial penalties. Different reports measure this cost differently, but the scale is consistent. A 2025 survey of 1,700 IT professionals found a median cost of \$2 million per hour for high-impact outages [1].

A separate 2026 industry survey found configuration or change management failures cause 41% of networking outages, ahead of hardware failure and third-party provider failure, which are roughly tied for second [2]. No published breakdown isolates cost to faults inside a single operator's own EVPN control plane specifically. These numbers cover networking outages in general. They give a sense of scale, not an exact figure for the fault this work addresses.

An inter-domain fault happens between two separately operated networks. One network might wrongly claim an address range that belongs to another, or a route might leak somewhere it should never reach. An intra-fabric fault happens entirely inside one operator's own network, with no other operator involved at all. Both are control plane problems. The difference is only whether the fault crosses an operator boundary. Tools exist for the inter-domain case. Graph neural network models trained on public BGP route collectors detect large-scale route hijacks between separately operated networks [3]. No comparable tooling exists for the intra-fabric case.

To the best of our knowledge, no published research detects and explains intra-fabric EVPN control plane faults in human readable form. Faced with this gap, network engineers are left with manual inspection. The only complete record of what happened sits inside Packet Capture (PCAP) files, byte-level logs of every message that crossed the network at the point where capture was running. A single fault can produce hundreds of thousands of messages spread across multiple files. Engineers open these files in tools like Wireshark, filter for relevant messages, and trace what happened by hand. Wireshark only displays the packets. Each message gets checked against the engineer's own knowledge of correct protocol behaviour, drawn from two sources. The first is an operator's own internal runbooks, documenting past incidents. These are not published and cannot be reused here. The second is the published Request for Comments (RFC) documents that define the protocol itself. RFC text states what normal control plane behaviour looks like, what a specific fault looks like on the wire, and what deviations are not permitted. PCAP2STORY relies on RFC text for this reason and is built as an automated detection and explanation pipeline. It takes raw packet captures as input and uses a language model to produce a human-readable explanation of exactly what happened, with a timestamp, the node involved, and the specific RFC rule that was broken. It suggests a next step for the engineer to check.

1.2 Hypothesis

A pipeline reporting that something failed leaves an engineer with the same manual investigation **PCAP2STORY** is meant to remove. Each detection rule's trigger condition is written directly from the RFC clause defining correct behaviour for that fault. The explanation stage then states which of these clauses the fault contradicts.

PCAP2STORY's hypothesis is that **EVPN control plane faults can be reliably detected directly from raw packet captures**, and that **each detected fault can be explained in a way directly traceable to the specific protocol rule it violates**, rather than presented as an unexplained failure. Both parts of this hypothesis were confirmed. Detection matched every ground-truth label without a single false positive, excluding scenarios structurally undetectable from packet captures alone. Explanations were generated by a language model and checked five separate ways, detailed in Chapter 5. Where a defect was found, correction resolved most of them, with the correction method used in the final pipeline chosen for the reliability of its convergence, not simply the number of files it resolved.

1.3 Contributions

Reliably detecting and explaining EVPN control plane faults from raw packet captures required four distinct pipelines.

Dataset generation - A pipeline that produces labelled packet captures, mostly from a real containerlab testbed, with a synthetic path for scenarios that could not be captured directly. No comparable public dataset for this fault class currently exists.

Event extraction and fusion - Turns raw captures from each vantage point into a timestamped event stream. Merges these separate streams into a single fused timeline, recognising the same underlying event when seen at more than one vantage point.

Detection - An RFC grounded pipeline of rules that checks the fused timeline against a fixed trigger condition per fault type, identifies and categorizes the fault, and resolves cases where more than one rule fires for what is really one underlying event. It generalizes across two topology sizes without any node or peer relationship hardcoded.

Explanation - An RFC grounded retrieval-augmented generation pipeline that turns a detected fault into a plain-language summary of what happened in the capture, grounded in the specific RFC rule violated, suggests a next step for the engineer to check, and corrects its own claims when a defect is found. Output quality is evaluated separately in Chapter 5.

The full implementation, including dataset generation, detection, and explanation, is available on [GitHub](#).

1.4 Thesis Structure

Chapter 2 introduces BGP and EVPN fundamentals, defines the seven fault types in scope, and reviews related work in BGP anomaly detection, log-based fault diagnosis, and LLM-grounded explanation, positioning **PCAP2STORY** against it directly.

Chapter 3 describes how the real and synthetic packet capture corpus was generated.

Chapter 4 covers **PCAP2STORY**'s three-stage architecture, event extraction and fusion, detection, and explanation, describing how each stage actually works and fits into the pipeline. Explanation itself covers retrieval, generation, and correction; correction compares three methods, Self-Refine, DISC, and Multi-Agent Debate, against the same defects.

Chapter 5 evaluates detection accuracy against ground truth and explanation quality across five independent checks, groundedness, atomic claim decomposition, self-consistency, judging by independent models, and discriminability across incidents, each targeting a different kind of failure.

Chapter 6 summarises the results, states the limitations they hold under, and outlines future work.

Appendix A explains each trigger mechanism and how it was engineered into a scenario. It also gives scenario counts per fault type and mechanism behind Chapter 3.

Appendix B covers the five rule modules not described in full in Chapter 4.

Appendix C gives a worked example of the explanation output for every fault type, including specific defects shown before and after correction.

Chapter 2

Background

2.1 BGP and EVPN Fundamentals

A router is a device that forwards traffic toward its destination across a network. BGP tells these routers which paths lead to which destinations. Two routers form a BGP session by opening a connection and exchanging routes, then sending regular keepalive messages to confirm the session is still alive. This connection runs over TCP, so a BGP session ending can also be visible as a TCP-level event: a reset or a connection close, on top of whatever BGP itself reports. If keepalives stop arriving in time, the session is considered down and its routes are withdrawn.

Figure 2.1 shows one of the two topologies used to generate packet captures, and the rest of this section refers back to it directly. The topology is kept simple to start, since no prior dataset or published work exists for this fault class, discussed in Section 2.3.

In larger networks, routers do not connect to every other router directly. Instead, a small number of routers act as a Route Reflector (RR). Each RR receives routes from many peers and passes them on. This way, a router only needs one session with the reflector, not a session with every other router. In Figure 2.1, RR1 and RR2 are the two reflectors, and PE1, PE2, and PE3 each hold one session with RR1.

EVPN advertises MAC addresses learned from customer devices. A device at one site can learn where a MAC address at another site lives, without the network needing to flood traffic across the whole network core to find it. Each site connects to this shared core through a PE. PE1 through PE5 in Figure 2.1 are these site connections. Most sites connect through a single PE. A site can also connect through more than one PE at once, for redundancy, keeping its MAC addresses reachable even if one PE fails. PE1 and PE2 do exactly this. They jointly serve one Customer Edge (CE) device, the

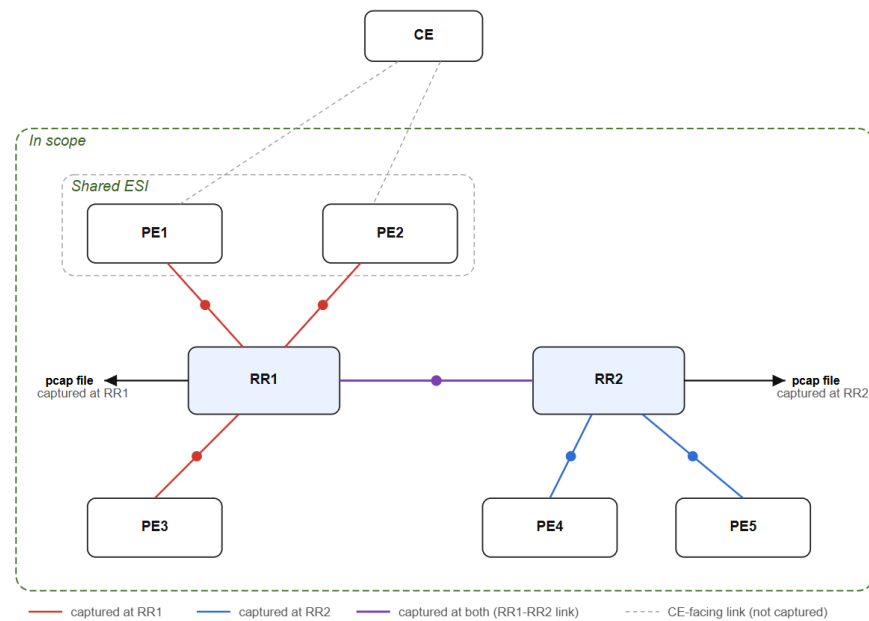


Figure 2.1: Topology of 5PE/2RR, used to generate Packet Captures

customer's own router or switch sitting outside the EVPN network itself. This shared connection is identified by an Ethernet Segment Identifier (ESI), shown by the dashed box around PE1 and PE2 in the Figure 2.1. Because more than one PE could otherwise forward traffic onto the same segment at the same time, the PEs sharing an ESI elect one of themselves as the Designated Forwarder (DF). Only the DF is allowed to forward that traffic until the next election. A new election takes place whenever a segment's membership changes, PE1 and PE2 in this case.

Packet captures are collected at all RRs, not at the PEs. In the topologies used, every PE peers with exactly one RR, so capturing at RR vantage points gives a complete view of control plane traffic without a capture process running on every PE. This does not necessarily hold for a topology where a PE connects to more than one RR directly.

Under normal operation, all of this happens continuously in the background. Sessions stay up, routes stay advertised, one PE stays DF for each segment, MAC addresses stay associated with the correct PE, each route keeps the identifying values that were configured for it, and no two PEs end up sharing a value meant to be unique. Together, these are examples of a steady state, the network's normal condition when nothing has gone wrong. A fault occurs whenever any part of this steady state breaks.

2.2 Fault Types in EVPN Deployments

Section 2.1 described what an EVPN network looks like when everything works. This section defines the targeted failure modes. Each fault type is a way steady state can fail.

Once a session is established, the two routers exchange UPDATE messages about which destinations they can reach. An advertisement tells the peer that a destination is now reachable through the sender. A withdrawal tells the peer that a destination the sender previously advertised is no longer reachable, so the peer must remove it. A session can also close through an explicit NOTIFICATION message, rather than through a silent keepalive timeout. A NOTIFICATION carries an error code and a more specific subcode explaining why the session is closing, for example an administrative shutdown rather than a failure. Bidirectional Forwarding Detection (BFD) tracks four states for each link it monitors: AdminDown, Down, Init, and Up, and a change between any of these states signals a link-level event separate from BGP's own session state.

A fault leaves a pattern in the traffic, and different faults leave different patterns. The same underlying fault can leave different signals on the wire depending on which mechanism triggered it, for example a link failing abruptly versus a link failing gradually as expected messages simply stop arriving. A burst of repeated withdrawals and re-advertisements can also appear, for example when a session drops and its routes are pulled, or when a route stops being uniquely identifiable. A deliberate shutdown produces an explicit notification message carrying a specific error code and subcode. The fault descriptions below are written in terms of exactly this kind of signal, since it is what the detector actually looks for on the wire.

Link Down - A session between a PE and its RR stops working, through any of the mechanisms described above. The visible effect is the same regardless of cause, the session drops and its routes are withdrawn.

RR Down - An RR itself stops responding, typically caused by a software crash or an administrative shutdown of the RR process. Every PE that depends on that RR loses its session at once, since the RR is the shared point every one of those PEs relies on.

PE Cease - A PE deliberately closes its own session, usually triggered by planned maintenance or an administrative shutdown command, rather than the session failing on its own. This is an administrative action, not a network failure, but it produces a similar visible effect to Link Down, so distinguishing the two matters.

Route Target Misconfiguration - Every route carries a Route Target, a tag that controls which PEs are allowed to import it. This fault is caused by a Route Target configured incorrectly on one or more PEs, either an explicitly wrong value or a fallback default that does not match the network's intended import and export policy. The PE then either accepts routes it should not, or rejects routes it should accept, with no visible change in session state.

Route Distinguisher Collision - Every route also carries a Route Distinguisher, a value that keeps otherwise identical routes from different PEs distinct from each other. This fault is caused by two PEs being mistakenly configured with the same Route Distinguisher, so their routes can be confused for one another.

MAC Mobility - A MAC address moves from one PE to another, and the network has to update which PE is now responsible for reaching it. A single move that settles is not necessarily a fault, that is exactly what genuine device movement looks like on the wire. It becomes a fault when the address moves repeatedly or flaps back and forth between PEs without settling, or when a shared Ethernet Segment's normal dual-PE reachability is mistaken for a move that never happened. A MAC address reachable through both PE1 and PE2 in Figure 2.1 at the same time is not a fault on its own, since that is exactly what a shared Ethernet Segment is supposed to allow.

ESDF Toggle - The Designated Forwarder role for a shared Ethernet Segment changes unexpectedly, fails to change when it should, or fails to resolve at all during an election. Since only the DF is meant to forward certain traffic onto a segment, an incorrect, stuck, or unresolved DF state can mean traffic is dropped or duplicated.

Appendix A.1.1 and Appendix A.2.1 list the specific trigger mechanisms covered for each fault type. Chapter 4 shows how the detector's rule modules use these signals to identify and categorize a fault.

Many fault types can occur in an EVPN deployment. To start, this work focuses on a small set that our industry partner confirmed are common in practice. Several other fault types are known to occur, VLAN misconfiguration, MTU mismatches, and split-horizon filtering errors among them, but these were left out of scope.

2.3 Related Work

Inter-domain BGP anomaly detection - A graph neural network trained on BGP traffic from public route collectors (RouteViews, RIPE RIS) reports accuracy above 90% for large-scale anomalies such as route hijacks and malware-driven route instability, trained on anomaly events spanning from 2004 to 2021 [3]. A time-series decomposition model applied to BGP UPDATE traffic reports F1 scores above 90% on the same class of anomaly [4]. A statistical threshold-based model for BGP anomaly detection has been evaluated against named historical incidents, including a 2005 power failure at a Moscow substation that took a major Internet exchange point offline and produced a visible surge in BGP route churn [5]. More recently, Radosa represents the routing system as a knowledge graph and trains a graph autoencoder on observed route propagation patterns

to detect anomalies, validated against real-world routing incidents collected between 2008 and 2023 [6]. The same knowledge graph then grounds an LLM’s root-cause analysis, whose conclusions adjust the detector’s thresholds and retrain it incrementally over time. BEAR takes this further, using an LLM to generate a full natural-language report explaining a detected BGP anomaly directly from the raw BGP data, reporting 100% accuracy against Chain-of-Thought and in-context learning baselines [7]. This shows the field moving toward LLM-based explanation alongside classical detection. Commercial tools follow the same focus, Kentik and ThousandEyes watch BGP for hijacking and route leaks in real time, at a cost of tens of thousands to several hundred thousand dollars a year [8, 9, 10, 11]. Academic and commercial approaches alike converge on the same target, anomalies between separate networks.

Learned detection across adjacent domains - Log-based anomaly detection has relied on reconstruction and prediction-based methods across nearly a decade of published work. DeepLog predicts the next expected log event from sequence history and flags large deviations [12]. LogBERT masks tokens within a log sequence and flags poor reconstruction [13]. LogLLaMA replaces DeepLog’s LSTM with a larger pretrained language model but keeps the same next-event prediction logic [14]. LogLLM instead pairs a BERT encoder with a language model classifier, extracting semantic vectors from log messages and classifying the resulting sequence directly [15]. Root cause analysis goes one step further, locating which component caused a fault rather than only flagging. Earlier causal-graph methods build this structure offline from a full batch of historical data before any detection can begin; CORAL instead updates the causal graph incrementally as new data arrives, avoiding retraining from scratch [16]. PRISM abandons causal-graph construction entirely, arguing it is often unreliable in practice, and ranks root causes by comparing internal and external anomaly signals directly [17].

LLM-based fault diagnosis and explanation - A separate body of work applies large language models to network fault diagnosis and explanation. PROBE runs an ensemble of language models directly over raw 802.11 (Wi-Fi) packet captures, each producing an independent draft diagnosis that a reconciler model then scores for evidence support [18]. Its own evaluation finds that self-reported confidence clusters at 0.95 regardless of actual diagnostic difficulty, with 71% of values landing exactly on that number, meaning confidence score carries little information about whether a given diagnosis is actually reliable. PropLLM assumes an anomaly has already been flagged upstream and performs hop-by-hop causal backtracking through a knowledge graph to localise root cause, reducing the hallucination rate by 50.8% over its strongest baseline

[19]. TN-AutoRCA reasons over expert-verified knowledge graphs built from telecom alarm data and iteratively repairs systemic flaws in a generated diagnosis across several rounds, more than doubling baseline LLM performance on its own benchmark [20].

A distinct group of systems detect first, then explain, across several domains. MSADM detects anomalies in heterogeneous device metrics using a trained model, then converts the result into a natural-language report with mitigation guidance via an LLM [21]. Holmes takes a similar shape in cloud DDoS investigation. A lightweight classifier triages traffic first, and only on a flagged anomaly does an LLM agent examine extracted packet evidence, constrained to quote verbatim substrings from that evidence rather than generate freely [22]. Isaac et al.'s 5G Core Fault Analysis Engine detects faulty frames directly from 5G core packet captures using a trained classifier. It then generates troubleshooting guidance through an LLM grounded in 3GPP specification documents, the telecom industry's standards body [23].

A related set of systems also operates on raw packet captures directly. PCAPVision classifies telecom call-flow failures from PCAP data using computer vision, converting packets into images for a CNN to label as failed or successful [24]. LLMcap flags anomalous PCAP segments using a self-supervised language model trained with masked language modeling, with no labelled data required [25]. Llama-PcapLog fine-tunes Llama-3 jointly on raw packet captures and system logs to answer security questions and generate remediation code [26].

RFC-grounded LLM output - A narrower strand of work grounds language model output in RFC text specifically, though for a different artifact than fault diagnosis. RFCAudit uses an LLM agent to audit protocol implementation source code against RFC specifications, identifying functional bugs where an implementation diverges from its specification [27]. A related line of work uses RFC text as the object of analysis rather than the grounding source, checking specifications for internal inconsistency rather than checking an artifact against them [28].

Evaluating LLM output quality - Most recent work applies LLMs for explanation, and evaluating the reliability of that output has become its own active line of research. Self-Refine shows that a LLM can improve its own output through iterative self-critique without additional training [29]. More recent work separates the correction step from the checking step entirely, using an independent verifier and judge before any rewrite is allowed [30], or coordinates multiple models through structured critique and judge-guided consensus specifically for fact verification [31]. The original LLM-as-a-judge study finds that a sufficiently capable LLM agrees with human evaluators over 80%

of the time when scoring open-ended output [32]. It also identifies systematic judge biases, including position and verbosity bias. A more recent survey formalises this into a full reliability framework for LLM judges, reflecting how quickly the field has moved from demonstrating judge agreement to systematically correcting for judge bias [33].

Semantic entropy clusters LLM outputs sampled repeatedly for the same input and measures entropy over the resulting semantic classes to detect confabulation [34]. A more recent variant needs only half as many sampled outputs to reach the same result, while improving detection accuracy [35]. Separate from either judging or entropy-based approaches, natural language inference models are used to check whether a generated claim is logically entailed by its source evidence, verification independent of any LLM’s own opinion [36]. Conformal prediction offers a different direction again, producing a statistically calibrated confidence interval around a judge’s score rather than a single point estimate of trust [37].

2.3.1 Research Gap

No prior work does all three things PCAP2STORY does together: detect and explain within one system, target EVPN control plane faults specifically, and ground the explanation in a written specification.

Detectors like the BGP anomaly systems and the adjacent-domain methods stop at flagging a fault, with no explanation. BEAR and PropLLM explain a fault but assume it was already detected elsewhere. MSADM, Holmes, and PROBE do both, but ground that explanation in a trained classifier’s output, quoted evidence, or agreement between models, not in RFC text. RFCAudit and RFCScope use RFC text, but for source code or the RFC’s own consistency, not packet-level traffic. None of this targets EVPN control plane faults. No labelled dataset for this fault class exists either, since none of the systems above needed one.

Every approach above has nothing external to check its claims against once the explanation is written, only the detector’s own verdict, quoted evidence, agreement between several models, or a knowledge graph, each of which is trusted rather than checked. PCAP2STORY’s explanation still draws on detector output and topology the same way, but each RFC-grounded claim in it can additionally be checked against the specific clause it cites, the same kind of check RFCAudit performs for source code, applied here to a detected packet-level fault instead. **PCAP2STORY** closes this gap. One system detects and explains EVPN control plane faults, and every claim in that explanation is checked against the RFC clause it comes from.

Chapter 3

Dataset Generation

3.1 Overview

PCAP2STORY generates data in two ways. The primary source is packet captures collected from a testbed. A testbed is a small network built specifically to run and test against. This testbed uses Free Range Routing (FRR), a real open-source implementation of the routing protocols this dissertation studies. This covers 369 scenarios in total: 293 with an injected fault and 76 without, used to test against false positives. A smaller synthetic corpus of 74 scenarios is generated without any routing daemon. It covers scenarios where testbed constraints made direct capture impractical, detailed in Appendix [A.3](#). Together, the two corpora give 443 labelled scenarios.

Every real or synthetic scenario is accompanied by a metadata file recording the fault type, the affected node, the trigger mechanism, the time of injection, and whether the scenario recovers. If it does, also records when. This is the ground truth the detector's output is checked against. Not every fault type uses this same recovery framing. **MAC Mobility** records `move_completed` and `move_completed_time` instead of `recovered` and `time_of_recovery`, since a move is a transfer between PEs rather than a session going down and coming back. A sample metadata file is shown in Listing [3.1](#).

```
{
  "fault_type": "Link Down",
  "event_affected_node": "PE1",
  "trigger_mechanism": "Hold Timer Expired",
  "time_of_first_fault": "2026-07-31T17:02:16.284147Z",
  "recovered": true,
  "time_of_recovery": "2026-07-31T17:02:32.135852Z"
}
```

Listing 3.1: Sample metadata file recording ground truth for a Link Down scenario

3.2 Real Captures

Real captures come from a testbed built with containerlab, a tool that deploys a network topology as a set of connected containers from a single topology definition file. This runs as a layered stack, shown in Figure 3.1. Windows Subsystem for Linux 2 (WSL2) provides the underlying Linux environment on the host machine. Docker Community Edition (Docker CE) runs natively inside that WSL2 environment, providing the container runtime. Containerlab sits on top of Docker CE, reading the topology file and using Docker to create and wire together the actual containers, one per PE or RR node. FRR runs inside each container as the routing software, with BGP, OSPF, and BFD as separate daemons, background processes that run continuously. Containerlab wires the containers together with virtual Ethernet links matching the topology’s specified connections, and starts each container’s FRR daemons automatically on boot.

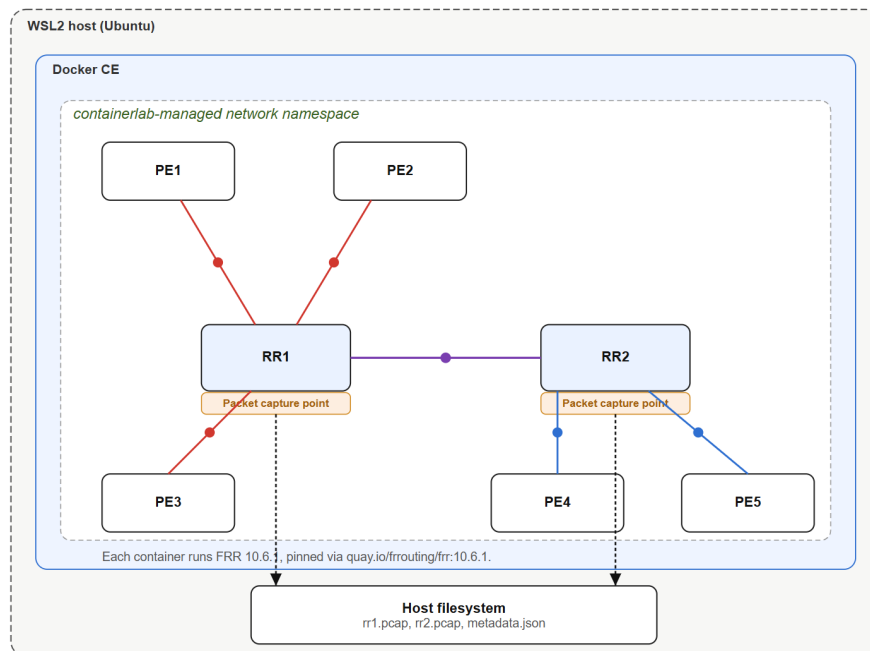


Figure 3.1: Testbed used to generate real captures

Two topologies were built this way: the five PE, two RR topology described in Section 2.1, and the larger ten PE, three RR topology shown in Figure 3.2. Both use the same FRR version across every node. Neither topology produces a working shared Ethernet Segment, since the configuration this requires never worked reliably on either testbed. Section 3.3 explains why.

Once deployed, each PE container’s BGP daemon peers with its assigned RR, and each RR’s BGP daemon reflects routes between its peers, exactly as described in Section 2.1. Faults are injected directly against the running containers, for example severing

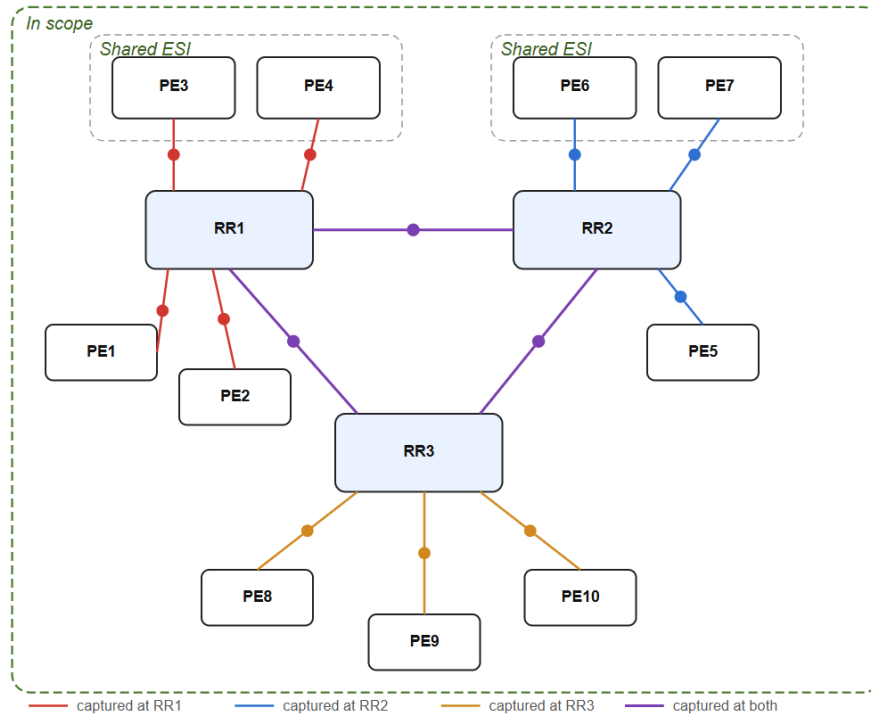


Figure 3.2: Topology of 10PE/3RR, used to generate Packet Captures.

a link, killing a routing process, or applying a deliberate configuration mismatch. Packet capture runs throughout on all RRs for the duration of each scenario, directly on the RR containers, and the resulting pcap files are written to the host filesystem once each scenario finishes. System time is recorded at the moment each fault is induced, and recorded again at the moment of recovery. These become time of first fault and time of recovery in the metadata file, saved to the same folder as the corresponding pcap files.

BFD and OSPF traffic, alongside BGP, appear in every capture. Section 4.2.1 explains which of this traffic is actually relevant to each fault type.

Each of the seven fault types from Section 2.2 is captured through more than one triggering mechanism where the underlying protocol allows it. For instance, Link Down is captured through three separate mechanisms: a BFD timeout, a BGP hold timer expiry, and a failed TCP connection, since each produces a different wire signature and the detector needs to handle all three. For each mechanism, both a recovered and a not-recovered variant were generated deliberately. This let the detector's ability to distinguish an ongoing fault from a resolved one be tested directly. It also let the explanation's output be checked against both cases, since a resolved and an ongoing fault should have different wording in the final explanation.

A further set of scenarios test the detector against more complex patterns: the same fault type recurring several times within one capture, two different fault types combined

in a single capture, or one fault with a varied recovery time. These check how the detector handles repeated, overlapping, or varied-timing faults. Full trigger mechanism descriptions and scenario counts are given in [Appendix A](#).

3.3 Synthetic Captures

Synthetic captures are generated programmatically rather than from a running network. No FRR routing daemon runs at all in this path. A Python script constructs each message directly, encoding the same BGP and EVPN wire format a real FRR session would produce byte for byte. The resulting capture is indistinguishable at the protocol level from a genuine session, even though no actual routing session exists. Each message field is set explicitly according to the relevant RFC clause, from route type and community values to sequence numbers. This gives the same RFC traceability as real captures, despite the different generation method.

This method was used for scenarios the testbed could not reliably capture, principally the ESDF Toggle fault type and the ES-Import Route Target mismatch variant. Both depend on a shared Ethernet Segment, which the testbed could not establish reliably on either topology, detailed in [Appendix A.3](#). It also allowed precise timing control the real testbed could not reproduce, used for MAC Mobility’s rapid and repeated flap sub-cases, which require exact, repeatable timing between moves to distinguish a genuine fault pattern from ordinary background churn.

Full generation methods and scenario counts are given in [Appendix A](#).

3.4 Scope Boundaries

Two mechanisms are handled differently from the rest of the corpus. One produces a capture indistinguishable from normal traffic. The other can never show recovery within a capture at all. Both are structural properties of the fault itself, not gaps in the dataset.

The import-only Route Target mismatch, detailed in [Appendix A.1.1](#), only changes how a PE filters routes it receives, not what it transmits. So the fault leaves no trace in what was actually sent. All 30 scenarios using this mechanism are included in the corpus.

An RR Down mechanism, a full container-level kill rather than a daemon-level kill, was excluded from the corpus entirely. Its recovery requires the entire network fabric to be redeployed, which necessarily happens after packet capture has already stopped. Recovery for this mechanism can never appear inside a capture, regardless of capture window length.

Chapter 4

System Design and Implementation

4.1 System Overview

PCAP2STORY takes raw packet captures from multiple vantage points (RR), and turns them into a human-readable summary of the incident, including what happened and what to check next. Figure 4.1 shows the full pipeline with three stages: event extraction and fusion, detection, and explanation.

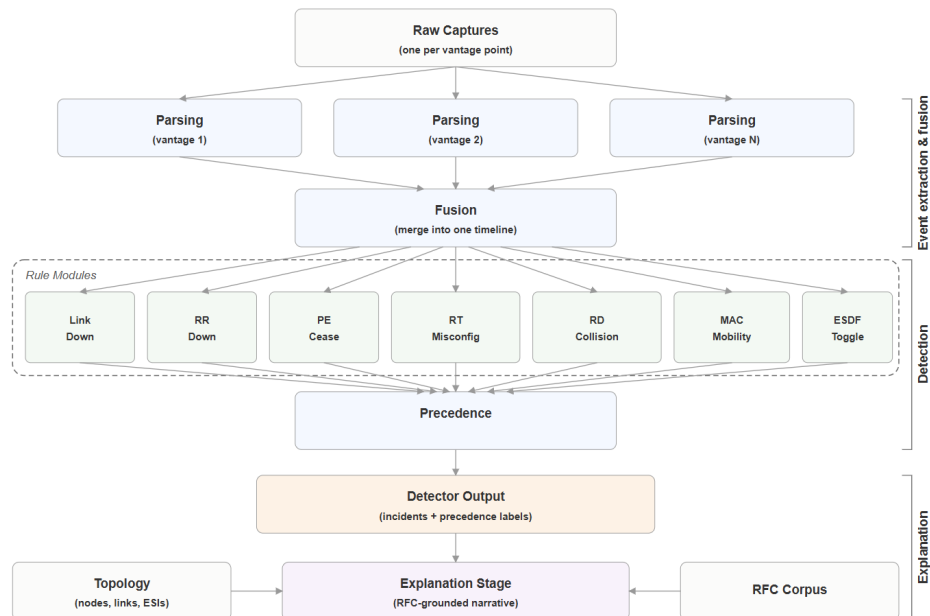


Figure 4.1: End-to-end pipeline

Each vantage point captures the same network independently, so the same real event can appear once at each RR that saw it. Event extraction and fusion resolve this first. Each raw capture is turned into a timestamped event stream. Streams from different vantage points are then merged into one timeline, so the same fault is not counted twice.

This fused timeline is checked against a set of RFC-grounded detection rules, one per fault type, to identify what went wrong. Because these rules run independently, more than one can genuinely fire for what is really a single underlying fault. For example, an RR going down looks like several separate link failures to the rules watching for those. Precedence resolves this next, using timing and topology to tell a genuine multi-fault capture apart from one fault whose effects triggered several rules at once.

Detections pass through precedence, then reach explanation. This is the stage that produces the human-readable output. A language model writes this output, drawing on the detector's findings, the network's topology, and RFC text retrieved for the fault rather than relying on what it already knows. This retrieval is structured as a knowledge graph, a network of linked facts that lets related information be found through its connections rather than by matching text alone. It is built from the RFC text itself, so it reflects what the specification says rather than a pattern learned from past behaviour.

4.2 Stage 1: Event Extraction and Fusion

4.2.1 Event Extraction

This is the first stage in Figure 4.1, taking the raw packet captures from Chapter 3. Each capture is independently parsed, turning relevant packets into a stream of timestamped events. Every event carries a timestamp, the vantage point that captured it, an event type, the node it involves, and a set of fields specific to that event type. A route advertisement or withdrawal carries the Route Target, Route Distinguisher, MAC address, IP prefix, ESI, and Ethernet Tag, which identifies the broadcast domain a route belongs to. Where relevant, it also carries the Designated Forwarder election state or MAC mobility sequence number. A session teardown is signalled by a NOTIFICATION message, which carries the error code and subcode explaining why. A BFD state change carries the new state and a diagnostic code, stating the reason for the change. These are the fields checked to decide whether a fault has occurred, described further in Stage 2.

A healthy BGP session sends constant keepalives. Only the first one after a session opens is kept, since it confirms the session came up. Every keepalive after that carries no new information while the session stays healthy. When keepalives actually stop arriving, that failure does not need to be inferred from their absence. It surfaces directly as its own event, a hold timer expiry or session teardown, which is already captured. OPEN, UPDATE, and NOTIFICATION messages are always kept, since this is where a fault actually shows up on the wire. A TCP reset or connection close is also kept, but

only when it occurs on BGP's port. This signals the underlying session has ended, not a TCP event on some other unrelated connection.

OSPF produces Hello messages between neighbors as part of normal operation. None of PCAP2STORY's fault types affect OSPF state, so OSPF traffic carries no relevant signal and is discarded. BFD state changes are kept, since a transition between any of the four states described in Section 2.2 signals a link-level event. Repeated BFD reports of unchanged state, ROUTE-REFRESH messages, which ask a peer to resend its routes without any actual change in reachability, and any BGP or OSPF content beyond what is described above are all discarded, since none of it carries information relevant to any of the seven fault types.

Real captures can occasionally include a malformed or truncated frame. When this happens, extraction simply discards that specific frame and continues, rather than stopping. Synthetic captures, generated directly by script, do not have this issue. BGP messages can be split across multiple TCP packets or bundled together in one, so the parser reconstructs actual message boundaries from the raw TCP byte stream before extracting events. Packets on BGP's port carrying no payload, ordinary TCP acknowledgments with no BGP data are dropped before this reconstruction step.

4.2.2 Fusion

When RR1 reflects a route to RR2, shown in Figure 2.1 both capture their own copy of what is, in reality one advertisement or withdrawal. Left uncorrected, this would be counted as two faults instead of one. A link or session event duplicates for a different reason. When a TCP reset travels directly between two RRs, each RR's own capture can record it independently, producing two copies of the same underlying signal. Fusion exists to recognise both cases as a single event.

Fusion handles the two cases differently. A route advertisement or withdrawal is matched by the node involved, the event type, and the route's identifying fields: Route Target, Route Distinguisher, MAC address, IP prefix, ESI, and MAC mobility sequence number where relevant. IP address is deliberately excluded from this match, since a reflected route can be readdressed at each hop, and matching on IP would break the link between the original and reflected copies. A link or session event is instead matched by content plus the pair of IP addresses involved, since it is tied to one specific link rather than something that changes address as it travels.

A fixed time window is needed alongside content matching, since two genuinely separate faults could otherwise share the same identifying fields and get wrongly

merged. The same reasoning applies to the fixed windows used throughout Detection (Section 4.3). Events matching on content are only merged if they also fall within 500 milliseconds of the first event in their cluster. This window was set to fall between the two observed clusters: simple reflected events, which land within tens of milliseconds, and recovery-driven re-advertisements, which can reach up to 1.10 seconds. No real event being seen twice in the corpus falls between roughly 50 and 500 milliseconds, so any threshold in that range separates the two cases equally well. Reflected updates and shared-link events land within milliseconds of each other in the real captures measured, while two unrelated events of the same type happening seconds or more apart are not merged. Events whose type cannot be reliably matched this way are never merged, each stays its own cluster.

Once a cluster is formed, the fused event keeps every field from only the earliest-observed copy in that cluster. Fields are not combined field by field with the other copies in the cluster. In the current topologies, every PE peers with exactly one RR, so the earliest copy is always the original advertisement, never a later reflected one.

A vantage point is authoritative for a node if the network's topology places it in a position to see that node's traffic directly, rather than only through reflection. Fusion records both how many vantage points saw each event and whether any of them were authoritative, since this helps confirm which node is actually the affected one when a detection is later reported.

4.3 Stage 2: Detection

4.3.1 Rule Modules

This is the detection stage in Figure 4.1, running a set of rule modules against the fused timeline that Section 4.2.2 produced. Each module's trigger condition was written by hand from the RFC clause defining correct behaviour for that fault type.

RT Misconfiguration - Route Target semantics are defined in RFC 4360. The module searches the fused timeline for a BGP UPDATE whose Route Target differs from the value expected for the advertising PE. Every node the topology defines with a PE role is checked this way, with the expected value read directly from that node's own configuration. A mismatched advertisement is grouped into an incident and removed from consideration. The search then runs again on what remains, so a file with more than one independent misconfiguration produces more than one incident.

A deviant Route Target found on a Type-4 Ethernet Segment route is classified as

ES-Import, since that route type carries an RFC 7432 field called the ES-Import Route Target, in addition to the regular Route Target. A deviant Route Target found on any other route type is classified as Auto-Derived. Both mechanisms share the same fixed grouping window of around two seconds, keeping further deviant advertisements from the same PE within that window in the same incident. A third mechanism, import-only, described in Appendix A.1.1, changes only how a PE filters routes it receives, never what it transmits. The absence of any wire-visible signal for this mechanism was confirmed by running the search against every import-only scenario, which found no deviant Route Target in any of them.

Recovery is detected when the same PE later advertises the correct Route Target for the same route. Route identity here has a related ambiguity to the one Fusion solves in Section 4.2.2, but for a different reason: two structurally different route types can look identical unless route type and Ethernet Tag are also matched. The output reports the misconfigured value found and the type of every route it appeared on. A sample detection output from a real auto-derived mismatch scenario is shown in Listing 4.1.

```
{
  "fault_type": "RT Misconfiguration",
  "trigger_mechanism": "Auto-Derived Mismatch",
  "root_cause_node": "XPE2",
  "time_of_first_fault": 1785607702.999933,
  "time_of_first_fault_readable": "2026-08-01T18:08:22.999Z",
  "recovery_status": "RECOVERED",
  "recovered_time": 1785607718.942441,
  "recovered_time_readable": "2026-08-01T18:08:38.942Z",
  "misconfigured_route_target": "65000:100",
  "affected_routes": [
    "Type-2 (MAC/IP Advertisement)",
    "Type-3 (Inclusive Multicast Ethernet Tag)"
  ]
}
```

Listing 4.1: Detection output for an auto-derived Route Target mismatch on XPE2.

RR Down - Session teardown is defined in RFC 4271. The module's defining signal is full peer breadth: every direct peer the topology defines for that RR must show a fault event within a short window. Breadth is what separates a genuine RR failure from an ordinary Link Down affecting a single PE. Peer identity itself comes from the topology's own declared adjacency.

Each mechanism leaves its own pattern on the wire. A daemon kill produces an abrupt TCP close with no warning. A graceful administrative shutdown instead produces

an explicit Cease notification. Once one pattern is identified, the module checks a two second window to confirm every peer of the RR shows the same pattern, since a genuine RR failure means every peer loses its session, not only the peer checked first. A daemon kill's real signature is under five milliseconds, faster than a graceful shutdown's. The window is kept at a general two seconds for both rather than fine-tuned per mechanism.

When one RR genuinely fails, the surviving RR on their shared link independently produces its own fault-shaped event, since it too has lost that one connection. Without excluding this, the surviving RR's own signal could be reported as a second, separate RR Down incident. Peer breadth tells the two apart, since only the genuinely failed RR loses every peer, not just the one shared link.

Recovery is only reported once every one of the RR's real peers has reconnected. The reported recovery time is the last, slowest peer's own reconnection. A partial recovery, where some sessions are back but others are not, is never reported as resolved.

The remaining five modules follow the same detection principle, an RFC-grounded trigger condition checked against the fused timeline. Each has its own distinguishing shape: Link Down's multiple trigger mechanisms, PE Cease's single unambiguous signal, RD Collision's colliding nodes rather than a single root cause, MAC Mobility's origin-destination pairing, and ESDF Toggle's four mutually exclusive sub-triggers. Full detail for all other modules is given in [Appendix B](#).

4.3.2 Precedence

Precedence runs between Detection and Explanation, as shown in [Figure 4.1](#), resolving the rule modules' output before it reaches the explanation stage. Its task is not simply to remove duplicates. Real networks can suffer more than one fault at once, so several scenarios in this dataset deliberately combine more than one fault type within a single capture, and a single capture can produce detections from more than one rule module at once. When this happens, none of the seven modules record a causal link back to another module's detection, so timing is the only evidence available to tell them apart. Some overlapping detections are genuinely two separate faults occurring close together, and collapsing those would hide a real event. Others are one fault whose downstream effects trigger a second module by coincidence of timing, and reporting both would overstate what actually happened. Each rule below resolves one specific pair of modules that can legitimately overlap, using timing evidence measured from real captures.

RR Down versus Link Down - A single link failing at one peer and the RR itself failing both surface as a BFD or TCP session collapse at onset. The RR Down module's

own acceptance logic already requires full breadth: every one of the RR's direct peers must show a qualifying fault before an RR Down is accepted. Each individual peer session drops the same way whether one link failed or the RR itself went down, so checking peers individually cannot tell the two apart. Only checking all of the RR's peers together reveals which one happened. This rule checks whether exactly one Link Down incident co-occurs among the RR's peers within 1.5 seconds of the RR Down detection, and demotes the Link Down to an artifact if so.

RR Down Recovery Cascade - An RR Down failure eventually recovers, and its peers reconnect. This reconnect activity would be counted as a new, independent Link Down or PE Cease incident rather than a downstream effect of the RR's own recovery. This rule checks whether a Link Down or PE Cease incident occurs within 2.0 seconds after the RR Down's recovery time. If it does, that incident is marked as an artifact of the RR Down recovery rather than a genuine fault on its own.

PE Cease versus Link Down - A PE going through an administrative shutdown and a PE whose link has already failed can produce overlapping signals on the same node. This rule resolves PE Cease and Link Down detections that share the same root cause node. Its logic is the reverse of the RR Down versus Link Down rule above: there, timing close together marks one detection as an artifact, but here, timing close together is what confirms both are genuine. An administrative shutdown is a deliberate, slower action, so a Link Down and a PE Cease landing close together on the same node are more likely two views of the same real event than a false trigger. When the two occur within 1.0 second of each other, this rule marks both genuine. When the gap is larger than 1.0 second, the PE Cease is instead marked as an artifact of the earlier Link Down, a delayed side effect rather than a new, separate cease event.

MAC Mobility and ESDF Toggle Co-occurrence - A MAC address move and a Designated Forwarder re-election on the same Ethernet Segment can occur close together in real captures. RFC 7432 Section 8.5 ties Designated Forwarder election to Ethernet Segment membership, not to individual MAC moves. When both appear near the same time, a shared upstream event on the segment is the more likely explanation. Neither detection is a false positive, so this rule marks them as a confirmed co-occurrence instead of demoting either. The rule searches for a matching Designated Forwarder toggle against each MAC Mobility incident, using a 5 second window throughout. It looks first for a toggle shortly after the move on the same origin PE. If nothing matches there, it looks backward in time on that same PE. If still nothing matches, it looks at either PE sharing the Ethernet Segment, in either direction.

Link Down or PE Cease and Designated Forwarder Re-election -When a PE holding the Designated Forwarder role for a multihomed Ethernet Segment fails, through either Link Down or PE Cease, the remaining PEs on that segment must re-run election. RFC 7432 Section 8.5 and RFC 8584 both describe this as a required consequence of the failing PE leaving segment membership, not an optional or coincidental one.

Each incident carries the status this precedence assigns: genuine, an artifact of another detection, or a confirmed co-occurrence, together with a short templated explanation built from the matched rule's own computed values: timing gap, node, window. Every status carries this explanation internally. Sample precedence output, for a real PE Cease incident on PE1, sharing a root cause with a co-detected Link Down incident:

status: CONFIRMED_COOCCURRENCE

reason: the Link Down incident's trigger occurred 0.000246 seconds after PE Cease's own trigger, on the same node (PE1), well within the 1.0 second window this rule requires for same-event confirmation.

4.4 Stage 3: Explanation

4.4.1 Retrieval

Explanation is grounded in a corpus of fifteen RFCs covering BGP and EVPN, rather than the model's own knowledge alone. This matters because a claim grounded in retrieved text can be checked against the specific clause it comes from, the distinguishing property behind PCAP2STORY's whole approach, stated in Section 2.3.1.

Each RFC is split into chunks at its own numbered section boundaries [38], so a subsection is its own retrievable unit rather than being merged into its parent section. More elaborate chunking methods showed no consistent advantage over simpler structural approaches, and the strongest average result came from a method whose score reflected only a small number of successful runs [39]. A chunk exceeding 2,373 characters, the corpus's own measured 75th-percentile chunk length, is split further into sequential segments. All segments share one citation label.

Grounding is retrieved through a knowledge graph built from this RFC text, where each RFC section is a node and each connection between two sections is an edge. Edges connect sections either by direct textual reference, one section citing another, or by shared concept, terms that co-occur across sections. A flat retrieval mode without the graph also exists, compared against the knowledge graph in Chapter 5.

Retrieval uses *all-mpnet-base-v2* sentence embeddings [40], which represent each chunk as a 768-dimensional vector. A smaller model, *all-MiniLM-L6-v2* [41], represent-

ing each chunk in 384 dimensions, was tried but failed to tell similar-sounding sections apart, for example confusing the ES-Import Route Target community with the more general Ethernet Segment Route section. The larger embedding space resolved this.

For a given incident, a query is built from its fault type and trigger mechanism, embedded the same way as every RFC chunk. Retrieval then runs in three steps. First, the graph seeds from whichever chunks score well against this query. Second, it expands outward from those seeds by up to two hops along the graph's edges. Third, every candidate in that expanded pool is reranked by cosine similarity against the query, a measure of how closely two embeddings point in the same direction. The top two candidates are passed into the prompt, extended to three when the second and third candidates score within 0.01 of each other rather than dropping one arbitrarily.

Retrieval is not expected to fail against a detected incident, since each fault type's trigger condition was itself written from the RFC clause it now retrieves. A detection with no corresponding RFC text in the corpus would mean a rule fired with no grounding for its own trigger condition, which cannot happen given how the rules were built.

4.4.2 Generation

Generation runs after retrieval has selected an incident's citations. Each prompt combines those citations with the detector's output for the incident, and a description of the network built from the topology file's link data. This covers each PE's route reflector attachment, its expected Route Target, Route Distinguisher values, and which PEs share an Ethernet Segment. The system prompt instructs the GPT-5 model [42] to use only the facts given and produce four labeled sections: SUMMARY, NEXT STEPS, RFC CITATIONS, and RFC GROUNDING. For an incident that has already recovered, the NEXT STEPS instruction is dropped entirely, and the model instead receives a shorter prompt covering only SUMMARY, RFC CITATIONS, and RFC GROUNDING, since there is nothing to recommend for a fault that has already resolved itself.

Output is parsed into the four labeled sections named in the prompt: SUMMARY, NEXT STEPS, RFC CITATIONS, and RFC GROUNDING. Separately, the model produces a recommendation followed by a CERTAIN or UNCERTAIN tag and a short justification, self-reported to indicate whether the evidence given fully supports the recommendation. This tag is not written inline by the model; it is parsed out of the raw response, and a CONFIDENCE line carrying it is appended after RFC GROUNDING, present only for incidents that have not yet recovered. A sample output for RD Collision incident between XPE3 and XPE4, a multihomed Ethernet Segment pair:

SUMMARY: RD Collision due to a shared Route Distinguisher between XPE8 and XPE9 (both clients of XRR3) began at 2026-08-01T18:21:56.744Z and had not recovered by the end of the capture.

NEXT STEPS: Reconfigure XPE8 and XPE9 so the MAC-VRF RD values are unique per PE, matching the expected values (XPE8 uses RD 10.0.0.18:2 and XPE9 uses RD 10.0.0.19:2), and verify via XRR3 that EVPN routes now carry distinct RDs.

RFC CITATIONS:

- RFC 7432 Section 7.9 (Route Distinguisher Assignment per MAC-VRF)

RFC GROUNDING:

- RFC 7432 Section 7.9 requires that each MAC-VRF have an RD and recommends using a Type 1 RD built from the PE's IP address plus a number unique to that PE. This construction yields per-PE uniqueness and lets the RD identify the MAC-VRF that is advertising the NLRI. The incident shows XPE8 and XPE9 sharing an RD, which contradicts the recommended per-PE-unique Type 1 RD usage; given the expected RD values (XPE8=10.0.0.18:2, XPE9=10.0.0.19:2), they should be distinct. By sharing an RD across different PEs, the RD no longer uniquely identifies the advertising MAC-VRF, producing the observed RD collision that persisted without recovery.

CONFIDENCE: CERTAIN -- the capture identifies a shared RD between XPE8 and XPE9 and the expected RDs are distinct, so correcting the RD configuration will resolve the collision.

4.4.3 Correction

Correction is the final step within Explanation, run after Generation produces a draft. It applies to every explanation generated, since even a well-grounded explanation can misstate what a cited RFC actually requires. Three correction methods were built to catch and fix this, each assigning the work of checking an explanation differently from the work of writing it. Chapter 5 compares all three against the same files to decide which one runs in the final pipeline.

Self-Refine follows the original method's design of using a single model for generation, feedback, and refinement [29]. The model used is GPT-5, and the loop is capped at ten iterations. The critique step receives the same context passed to generation, plus the current draft, and checks the draft against that context. A verdict of APPROVED or CORRECTED determines whether the loop continues, with a rewritten draft accompanying CORRECTED. Figure 4.2 shows this loop.

Denoising Iterative Self-Correction (DISC) separates checking from writing [30].

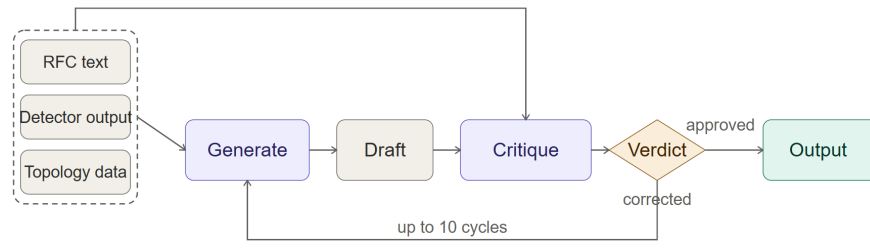


Figure 4.2: Self-Refine correction loop

A verifier generates targeted questions about the draft, and each question is answered independently against the same context passed to generation. A judge then reviews the answers and produces a verdict on whether the draft contradicts any of them. If it does, the draft is rewritten; if not, the loop ends. Judge, verifier, and corrector are three distinct model roles. A genuine contradiction has to be found before anything gets rewritten. GPT-5.6 acts as verifier and judge, GPT-5 as corrector. The loop is capped at ten cycles per file. Figure 4.3 shows this loop.

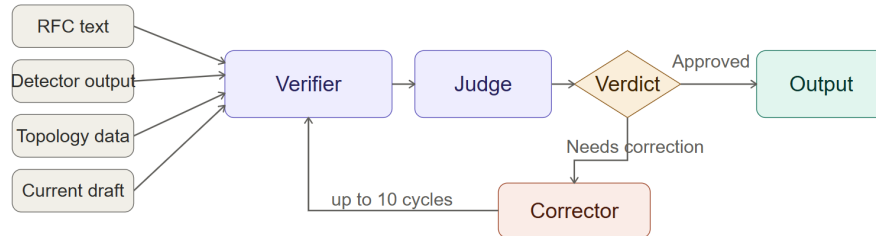


Figure 4.3: DISC verify-judge-correct loop

Multi-Agent Debate makes the check reciprocal: multiple models each propose a response and critique each other's reasoning to reach a final answer [43]. GPT-5 and GPT-5.6 each produce an independent draft and critique the other's, with GPT-5.6 producing a reconciled version. A judge then checks the reconciled draft against the same context passed to generation. If the check finds no remaining issues, the loop stops. Otherwise, both models critique the single reconciled draft directly, and GPT-5.6 reconciles again. This can repeat up to ten cycles per file. Each round of reconciliation is instructed to preserve every correction confirmed in an earlier round, so a later fix cannot silently undo one already made. Figure 4.4 shows this loop.

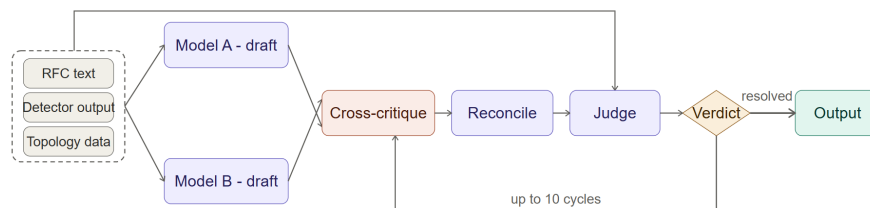


Figure 4.4: Multi-Agent Debate: draft, critique, reconcile.

Chapter 5

Evaluation

5.1 Evaluation Methodology

Detection is checked directly against a fixed protocol pattern. A rule either matches a fixed protocol pattern or it does not, so evaluating it is a direct comparison against ground truth, with no ambiguity in what correct means.

Explanation depends on a language model, so its quality cannot be checked against a single correct answer the way detection can. It is evaluated in two categories. The first checks whether an explanation is grounded, consistent, and accurate using independent methods that catch a different kind of failure. The second tests whether a defect found this way can actually be fixed, comparing the three correction methods described in Section 4.4.3 against twenty files, drawn from the full corpus of 443, with one file per fault type and trigger mechanism covering both recovered and not-recovered states.

5.2 Detection Accuracy

Detection was evaluated against the ground-truth labels recorded when each scenario was generated, achieving 100% agreement across all files. This excludes 30 files whose fault mechanism, an import-only Route Target mismatch, is structurally undetectable by design. Detection was also run against all 76 normal captures, real traffic with no injected fault, and produced zero false positives, a result that follows from the RFC-grounded nature of the detection rules. Each rule's trigger condition matches a fixed protocol pattern defined by the RFC itself, not a pattern learned from these specific files the way a trained model's would be. Several of its timing thresholds were themselves tuned against this corpus though, discussed further in the Section 6.2.

One caveat applies to recovery timing specifically. Ground truth records the time

recovery began. The detector reports the time when recovery was confirmed complete. The two can differ by roughly 40 seconds even when fault type and the fact of recovery are both detected correctly. Detection takes about five seconds per file on average.

5.3 Explanation Quality

The rest of this chapter evaluates a different question: whether the language model's explanations are actually correct, and how reliably that correctness holds.

5.3.1 Ablation Study

This section determines which setup the rest of this chapter should use, by testing retrieval mode and topology inclusion independently against a separate sample of thirty-five files. Precedence is not included here, it is already evaluated in Section 4.3.2.

Grounding and Retrieval Mode - Two questions are tested here: does using RFC text at all change the explanation, and does the retrieval mechanism (knowledge graph versus flat) itself, make a further difference. Each file's explanation was generated three separate times, once with no RFC retrieval, once with flat retrieval, and once with knowledge graph retrieval. No RFC retrieval meant no citations at all. Both retrieval modes produced citations. Retrieval changed NEXT STEPS in three ways, not just citations. In some files, it added a diagnostic step the no-retrieval version never included. In others, it dropped a fallback or check that only the no-retrieval version had included on its own. And even when flat and knowledge graph retrieval agreed on which RFC sections to cite, they still sometimes recommended different or contradictory actions.

Where flat and knowledge graph retrieval disagreed, each case was checked against the real facts and RFC text directly. Knowledge graph retrieval gave the more correct guidance in every one of these cases, and flat retrieval in none. This consistent advantage is why knowledge graph retrieval is kept as the system's default.

Topology - Removing the topology description tests whether it changes what an explanation recommends. Fifteen files were unaffected, since the fault had already recovered or the recommendation didn't depend on a configured value like a Route Target or Route Distinguisher. Among the rest, some changed only in wording, while others lost a specific value, an RD, RT, or ESI, that topology had supplied. SUMMARY lost the sentence explaining why two nodes were related, ESI partnership, RR attachment, while the underlying facts stayed correct. Citations and RFC grounding were unaffected by topology on every file. This confirmed topology is worth keeping in the prompt.

5.3.2 Groundedness Check

A groundedness check was run after every explanation is generated. It compares the stated fault type, root cause node, and recovery timing against ground truth, and confirms every cited RFC section was genuinely retrieved rather than invented. These checks catch fabrication and factual drift directly. The RFC-related overlap score is weaker by design: it measures shared vocabulary between the explanation and its cited passages, not whether the explanation represents what those passages actually argue.

This limit surfaced a real defect. Repeated generations of the same file consistently cited RFC 8584's coverage of individual access-circuit failures for a scenario the section does not cover. RFC 8584 Section 1.3.2 states that no procedure exists in the specification to trigger re-election for an individual access-circuit failure. Every repeated generation instead cited this section to explain a dual-PE failure, a materially different scenario the section does not address. The retrieved citation was identical across every repeat; the wording of each explanation varied, but the same wrong citation was reproduced without exception.

The cause was not a wording error in generation. The retrieval query built to search the RFC corpus was not distinguishing between a fault type's different trigger mechanisms, so the same section was retrieved and treated as the top citation for a mechanism it did not primarily describe. Once this cause was identified, every fault type and trigger mechanism combination in the corpus was checked the same way, comparing retrieved citations directly against the real RFC text.

Each affected query was corrected to use terms specific to its own trigger mechanism. Each correction was verified against the real RFC text before use, rather than relying on one shared phrase across every mechanism of a fault type. The corrected retrieval was re-run across the full corpus. The mischaracterization no longer appeared, and mechanisms retrieval already handled correctly showed no loss of accuracy.

One affected file, a Type-4 Ethernet Segment route withdrawal detected as an ESDF Toggle incident on the shared segment between PE1 and PE2, shows this concretely.

Before the fix, RFC CITATIONS: RFC 8584 Section 1 (Introduction)
RFC GROUNDING: RFC 8584 Section 1 establishes that the DF is elected from the multihomed PEs attached to an Ethernet Segment and that each participating PE advertises an ES route for that segment. PE1 and PE2 share the stated ESI, so the observed withdrawal of PE1's Type-4 ES route disrupted PE1's advertised participation in the segment's DF-election

control plane, corresponding to the detected ESDF Toggle.

After the fix, RFC CITATIONS: RFC 8584 Section 1 (Introduction); RFC 8584 Section 4 (The AC-Influenced DF Election Capability); RFC 8584 Section 2.1 (The DF Election Finite State Machine (FSM))

RFC GROUNDING: RFC 8584 Section 1 (Introduction) defines the Designated Forwarder (DF) role in EVPN as the PE responsible for BUM forwarding toward a multihomed CE on a given ES and Ethernet Tag, establishing why a DF state change on the shared ESI for PE1 and PE2 is operationally significant. RFC 8584 Section 2.1 (The DF Election Finite State Machine (FSM)) specifies that receiving an MP_UNREACH_NLRI for a previously received ES route (the LOST_ES event) triggers a transition from DF_DONE to DF_CALC. However, the facts do not establish that the Type-4 ES Route Withdrawal from PE1 was encoded as MP_UNREACH_NLRI for a previously received ES route or that the receiving FSM was in DF_DONE; therefore, LOST_ES is not asserted here. RFC 8584 Section 4 (The AC-Influenced DF Election Capability) states that the DF candidate list is built from the Originating Router's IPs in ES routes and must exclude any PE whose Ethernet A-D per ES (or per EVI for a given tag) has been withdrawn. In this incident, the ES route withdrawal from PE1 is not a Section 4 trigger; Section 4 candidate pruning is based on withdrawal of Ethernet A-D routes specifically, and the facts establish the observed ESDF Toggle until the DF state re-stabilized by 2024-03-11T07:01:16.697Z.

Before correction, retrieval never surfaced the section that actually names this withdrawal as a valid trigger. The gap was not a conflict inside the RFC itself, it was two compatible sections that never surfaced together until the query was corrected to distinguish between trigger mechanisms.

5.3.3 Atomic Claim Decomposition

Atomic claim decomposition tests at a finer grain: whether each individual factual claim inside SUMMARY and RFC GROUNDING is correct, not just whether the passage as a whole shares enough vocabulary with a real RFC section.

For each of the same twenty files, this check used one call to GPT-5.6, a stronger version of the GPT-5 model used for generation. It was given the file's SUMMARY and RFC GROUNDING text, the incident's ground-truth facts, and the RFC passages cited in that file. The prompt asked the model to first break the text into individual atomic claims, tagging each as either an incident fact (a node name, timestamp, fault type, or recovery status) or an RFC-content claim (a statement about what the cited specification requires), then check each claim independently and label it SUPPORTED,

UNSUPPORTED, or CONTRADICTED with the specific evidence for that verdict. Decomposition and checking are done within a single call rather than as separate roles, adapting the claim-decomposition step used to detect confabulation in generated text [34]. That method’s setting has no structured reference to check a claim against directly, so it estimates confabulation indirectly through entropy across resampled answers. Here, both claim types have a direct reference available: incident facts against the same ground truth used for detection accuracy, and RFC-content claims against the real RFC text cited. Each claim is checked against its reference directly.

The number of claims per file was not fixed, depending on how long and how multi-incident that file’s explanation was. Table 5.1 summarises results by fault type.

Fault type	Claims	Supported	Unsupported	Contradicted
Link Down	116	86	25	5
ESDF Toggle	83	61	18	4
MAC Mobility	80	66	13	1
RR Down	68	55	11	2
PE Cease	59	47	11	1
RD Collision	43	32	11	0
RT Misconfiguration	41	27	14	0
Multi-fault	46	41	5	0
Total	536	415	108	13

Table 5.1: Atomic claim decomposition results by fault type.

All of the contradicted claims were RFC-content claims; none came from an incident fact. Contradictions were concentrated entirely in RFC GROUNDING, and every one followed the same failure mode: a conditional, permissive, or scope-limited RFC clause restated as an unconditional or general requirement. ESDF Toggle accounts for a disproportionate share of these, 4 of 13 contradicted claims against 83 of 536 total claims, and its contradicted claims involve the same RFC 8584 route-type scope confusion, the retrieval defect corrected in Section 5.3.2.

5.3.4 Self-Consistency

Self-consistency tests whether the same incident, generated multiple times with no change to input, produces explanations that agree in meaning even when the wording differs. This is a check of internal agreement, not RFC correctness: whether two texts describing the same known incident say the same thing, regardless of wording. For each file, GPT-5 generated the same incident five times independently. This produced five

SUMMARY texts and five RFC GROUNDING texts per file, plus five recommended-action texts for incidents that had not yet recovered.

Each field’s five texts were compared pairwise, giving ten pairs per group. Three automated methods were tested against these pairs across the fifty-two groups. None of these three methods are language-model prompts asking for a judgment; each is a fixed scoring mechanism applied to the text. Each group was also manually compared for meaning equivalence, giving a reference to check each automated method against.

DeBERTa-MNLI checks whether one text follows logically from another, in both directions, then groups texts by which pairs pass this check. This is the same entailment-clustering mechanism used in semantic entropy [34]. It matched the manual comparison in only 24 of 52 groups. It never wrongly grouped texts that actually differed as if they agreed. Instead, it split texts that said the same thing into separate groups. This happened because passing the check between two texts does not guarantee it holds across all five at once. Longer passages, such as SUMMARY and RFC GROUNDING, gave more opportunity for this than the shorter recommended-action text.

Cosine similarity scores two texts by the angle between their embedded vectors. It uses *all-mpnet-base-v2* sentence embeddings [40], the same model already used for retrieval. It matched the manual comparison in 49 of 52 groups. Its three errors shared a pattern: consistently reworded text scored as differing in one multi-incident file, while genuinely shifted emphasis in RFC citation order scored as unchanged in two others.

BERTScore [44] compares two texts token by token using contextual embeddings, rather than one vector per text. It also matched the manual comparison in 49 of 52 groups. It caught one of the two citation-emphasis errors cosine similarity missed, but rated two runs of a MAC Mobility recommendation as equivalent when they actually recommended different corrective steps.

Both cosine similarity and BERTScore matched the manual comparison at the same overall rate. Cosine similarity’s errors involved missed differences in emphasis. BERTScore’s error missed a genuine change in the recommended action, a more consequential mistake in this setting. Cosine similarity matched the manual comparison most reliably of the three. By its measure, content stayed consistent across repeated generations in most groups. Wording varied more than meaning did.

5.3.5 LLM as Judge

An explanation’s RFC-grounded reasoning can share vocabulary with its cited source while still misrepresenting what that source argues, an error word-overlap scoring

cannot catch. This check audits explanations directly against the real RFC text to catch that specific failure, following the approach of using a capable model to score open-ended output [32].

Generation used GPT-5. Judging used two models: GPT-5.6, the same model family at a stronger version but within the same family, carrying some risk of intra-provider self-preference bias, and Gemini 3.1 Flash-Lite, a different provider entirely, included specifically as a cross-provider. The judge received the same context passed to generation, along with the generated explanation itself. It was prompted to flag any claim that misrepresents what the cited RFC text says. Each flagged issue required the specific wording and RFC passage as evidence. Two categories were used: **Meaning change** - a claim that contradicts or inverts the source, matching the established definition of intrinsic hallucination [45]. **Precision gap** - a claim that is directionally correct but drops a real qualifier or exact term, a distinct failure mode this evaluation defined on its own.

Table 5.2 summarises both judges' findings across the twenty files.

Judge	Total Files	Clean Files	Meaning changes		Precision gaps	
			Files	Instances	Files	Instances
GPT-5.6	20	3	14	17	14	26
Gemini 3.1 Flash-Lite	20	5	11	12	8	8

Table 5.2: LLM-as-Judge findings by judge model.

Most precision gaps softened a required-behaviour word such as **MUST** into vaguer general prose, dropped a scoping condition, or substituted a related but distinct term, for example "originating" for "advertising." These left the recommended action unchanged, since the underlying fact was still correct, just less precisely stated.

RFC 7432 Section 7.9 requires RD uniqueness only within one PE's own MAC-VRFs. One explanation instead stated that the RFC requires two different PEs to use different RDs, a broader claim the section does not make. In practice, two PEs following the RFC's recommended construction method, building each RD from a PE's own IP address and a PE-unique number, do end up with different values. That happens as a result of the construction method, not because the RFC states a cross-PE uniqueness rule. GPT-5.6 caught it in only one of the two files. Gemini caught it in both.

Gemini found fewer issues overall than GPT-5.6 in both categories. The two judges rarely agreed on which files were clean. One finding was caught identically and by both judges, on the same file. An explanation attributed a Designated Forwarder candidate-

list change to withdrawal of the Ethernet Segment route itself. RFC 8584 ties that pruning to withdrawal of a different route, the Ethernet A-D per Ethernet Segment route. A single judge's silence on a file did not reliably mean the file had no problem.

5.3.6 Correction Outcomes

Correction methods described in Section 4.4.3 were each tested against the same twenty files to see whether the defects found above could actually be fixed.

Self-Refine approved seven of the twenty files on the first pass. The remaining thirteen needed one or more corrections, averaging 1.7 critique calls per file. Every correction made was substantive: values were removed when not present in the incident facts, certainty was downgraded when the evidence only supported a qualified conclusion, and an RFC citation was rewritten from claiming a rule the section does not state to what the section actually requires, the same pattern as the RD Collision defect in Section 5.3.5. Self-Refine caught and corrected it on one of the two affected files, missing the identical error on the sibling file despite both receiving the same context and prompt, consistent with having no second model to catch what the first one missed.

DISC converged on nineteen of twenty files, reaching a judge verdict of no remaining issues. On the RD Collision defect, DISC caught and fully resolved it on both affected files, where Self-Refine had resolved only one. One file's correction initially appeared unresolvable. The detector's trigger was a Type-4 Ethernet Segment route withdrawal. Retrieval only ever surfaced RFC 8584 Section 4, which ties re-election to a different route. It never surfaced Section 2.1, which independently names a Type-4 route withdrawal as its own valid trigger. The retrieval query was corrected to surface both sections together. Re-run across every file, the fix held: no file regressed, and the previously unresolvable file converged.

Multi-Agent Debate reached full convergence on all twenty files. Both RD Collision files were resolved cleanly, each correctly scoping RD uniqueness to a single PE's own MAC-VRFs without asserting a cross-PE requirement the RFC does not state.

Self-Refine resolved the RD Collision defect only partially. DISC left one file unconverged, tied to the same retrieval gap corrected above, since it could not verify the draft against incomplete evidence. Multi-Agent Debate converged on that same file before the retrieval fix was applied, producing an answer despite the missing evidence. A method that always converges, even on incomplete evidence, is less trustworthy than one that withholds a verdict until it can be checked. DISC is the correction method used in the final pipeline on this basis.

A Worked Example - An RD Collision incident between PE3 and PE4 was generated with an initial claim overstating what RFC 7432 Section 7.9 actually requires.

Initial draft, RFC GROUNDING:

RFC 7432 Section 7.9 states the RD advertised in EVPN NLRI must be the MAC-VRF's RD and that RDs must be unique across MAC-VRFs on a PE; it further recommends using a Type 1 RD with the PE's IP address plus a per-PE-unique number. The expected RDs (PE3=10.0.0.13:2, PE4=10.0.0.14:2) follow this recommendation, whereas the detected shared RD between PE3 and PE4 deviates from it, creating an RD collision where distinct EVPN routes from two different PEs are not differentiated by RD.

Cycle 1 verdict: Mistake

Draft claim flagged: "whereas the detected shared RD between PE3 and PE4 deviates from it"

Reason: The draft presents sharing an RD across PE3 and PE4 as violating Section 7.9, but the verifier's own question confirms the cited requirement applies only within a single PE, not across different PEs.

Cycle 2 verdict: Mistake

Draft claim flagged: "when those routes carry a shared RD, they are not cleanly distinguished as separate origins, manifesting as the observed RD collision"

Reason: The draft attributes this consequence to RFC 4364 Section 4.3.6, but the verifier's answer states the cited excerpt does not establish that consequence at all.

Cycle 3 verdict: No Mistake, loop ends.

Final RFC GROUNDING:

RFC 7432 Section 7.9 states the RD advertised in EVPN NLRI must be the MAC-VRF's RD and that RDs must be unique across MAC-VRFs on a PE; it further recommends using a Type 1 RD with the PE's IP address plus a per-PE-unique number. The expected RDs (PE3=10.0.0.13:2, PE4=10.0.0.14:2) follow this recommendation; the detected sharing of an RD between PE3 and PE4 does not contravene the Section 7.9 uniqueness requirement (which applies within a single PE) but does depart from the recommended per-PE Type 1 construction, and that shared RD across PEs resulted in the observed RD collision where distinct EVPN routes from two different PEs are not differentiated by RD.

5.3.7 Discriminability Across Trigger Mechanisms

A discriminability check compares explanations across distinct incidents that share the same fault type and trigger mechanism. RFC-grounded content should stay similar

across these incidents, since the same clause governs all of them. Incident-specific content should vary, since the facts differ.

ESDF Toggle and MAC Mobility were excluded from this comparison. Several of their trigger mechanisms have fewer than ten scenarios each, as shown in Appendix A. The remaining five fault types gave eight fault-type and trigger-mechanism combinations. Every file was corrected using DISC, the correction method chosen for the pipeline per Section 5.3.6. Applied across the full corpus, 310 of 315 files converged. Each corrected explanation was split into its four sections: SUMMARY, NEXT STEPS, RFC CITATIONS, and RFC GROUNDING. Within each of the eight groups, every file’s text for a given section was compared pairwise against every other file’s text for that same section. Comparison used cosine similarity between *all-mpnet-base-v2* sentence embeddings. Table 5.3 shows the result.

Fault Type	Trigger Mechanism	n	Summary	Next steps	RFC Citations	RFC Grounding
Link Down	BFD Down	42	0.829	0.674	0.915	0.832
	Hold Timer Expired	32	0.842	0.603	0.980	0.879
	TCP RST	99	0.756	0.602	0.808	0.727
PE Cease	Cease/Admin Shutdown	84	0.753	0.684	0.767	0.693
RR Down	Cease/Admin Shutdown	11	0.814	0.648	0.820	0.737
	TCP RST	18	0.792	0.641	0.830	0.745
RD Collision	Shared RD	31	0.776	0.640	0.914	0.811
RT Misconfig.	Auto-Derived Mismatch	30	0.828	0.596	0.970	0.827

Table 5.3: Within-group cosine similarity across fault-type and mechanism combinations

RFC CITATIONS and RFC GROUNDING scored higher than SUMMARY and NEXT STEPS in every group. RT Misconfiguration’s RFC CITATIONS reached 0.986, the highest of any group. This fits a single RFC clause governing every occurrence of a narrowly scoped mechanism. The pattern held across group sizes ranging from eleven files to ninety-nine. Section 5.3.4 validates cosine similarity against manual comparison for this corpus, matching human judgment in 49 of 52 groups; the values in Table 5.3 are best interpreted against that validation rather than a fixed similarity cutoff.

Appendix C gives a worked example for every fault type and precedence scenario in this chapter, including the defects discussed above shown before and after correction.

Chapter 6

Conclusion

6.1 Summary

PCAP2STORY is built around two capabilities working together: detecting an EVPN control plane fault from raw packet captures, and explaining it against the specific RFC text it violates. No labelled dataset for this fault class existed either, so this work built one: real and synthetic captures spanning seven fault types.

PCAP2STORY's hypothesis was that EVPN control plane faults could be reliably detected directly from raw packet captures, and that each detected fault could be explained traceably against the specific rule it violates. Detection matched every ground-truth label across the corpus with zero false positives, excluding scenarios structurally undetectable from packet captures alone, confirming the first half of this hypothesis. Event extraction and fusion resolved the duplication that would otherwise occur when the same event is seen at more than one vantage point. Precedence correctly separated genuine multi-fault captures from a single fault whose effects triggered more than one detection rule.

Explanation quality was tested five separate ways: groundedness check, atomic claim decomposition, self-consistency, judging by two independent models, and discriminability across incidents, since explanation has no ground truth to check against the way detection does. Each targeted a different kind of failure, and each found something real. A dual-PE failure scenario was repeatedly explained using an RFC 8584 clause about single access-circuit failures, a section that does not actually cover it, until the underlying retrieval query was fixed. A separate explanation overstated what RFC 7432 actually requires, a generation-side error, and two independent judge models from different providers both caught it. Every explanation is grounded in retrieval, topology,

and the detector's own output, not the model's own knowledge. This grounding is what let both defects be traced to a specific, fixable cause instead of dismissed as unexplainable. This is the second half of the hypothesis confirmed: an explanation that can be checked against the evidence it claims to follow.

Most of the defects this evaluation found trace back to retrieval, not generation. The wrong RFC section surfaced as evidence, or the right section failed to surface at all, so the model was working from incomplete or incorrect material rather than failing to reason over correct material. This places more weight on the corpus and retrieval method than on the model that writes the final text. The Ablation Study supports this. Where the two retrieval modes disagreed on what to recommend, knowledge graph retrieval gave the more correct guidance in every case, checked directly against the real facts and RFC text.

Three correction methods were then compared, and the highest convergence rate was not what decided which one runs in the final pipeline. Multi-Agent Debate resolved every file, including one where the underlying evidence was genuinely incomplete. DISC stopped short on that same file rather than produce an answer it couldn't support, and this ability to withhold a verdict when evidence is incomplete is why DISC was chosen instead.

Among the systems reviewed, none combines all three of these: detecting from raw packet captures, targeting EVPN specifically, and grounding an explanation in a written specification. The results in this chapter close that gap directly, demonstrated by every defect found and corrected along the way.

Together, these results confirm both halves of the hypothesis: EVPN control plane faults can be reliably detected from raw packet captures, and each detected fault can be explained in a way that is directly traceable to, and checkable against, the specific RFC clause it violates.

6.2 Limitations

The results in this chapter hold under specific, stated conditions, listed below.

1. All captures come from a single testbed, running FRR as the only vendor routing software. Only two topology sizes were tested, not a wider range of deployments.
2. Some scenarios were generated synthetically rather than captured from a live network, since the testbed could not reliably produce a working shared Ethernet Segment. ESDF Toggle, some MAC Mobility and RT Misconfiguration variants exist only in this form, protocol-accurate but never validated against real network behaviour under load.

3. The dataset only contains the seven fault types and trigger mechanisms this project scoped in. Any other fault type was not generated or tested, and would go undetected against these rule modules, since a rule module only checks for the specific pattern it was written for.

4. Several precedence and detection rules use a fixed timing window, tuned to the specific testbed and files generated. A different topology, deployment, or network load could produce timing the window was not tuned for, and expose a window that no longer holds.

5. Not every EVPN control-plane fault can be identified from packet captures alone. Some mechanisms produce no wire signal at all. A packet-capture-based approach has no way to detect them, as discussed in Section 3.4.

6. The model's self-reported CERTAIN or UNCERTAIN tag cannot be treated as a reliable confidence signal on its own. Language model output is not fully consistent, even with identical input, so the same file was run repeatedly with identical facts and produced both tags across different runs.

7. An explanation, and any judge checking it, can only work with the information given to them. The judges in this evaluation were scoped to flag claims that are wrong, not claims that are missing. If a correct, supported point was simply left out of an explanation, no judge in this evaluation would catch that absence.

8. Generation was tested with only one model family, GPT-5 and GPT-5.6. The cross-provider comparison in LLM as Judge used Gemini 3.1 Flash-Lite alongside GPT-5.6, but only for judging. Generation itself was never tested with Gemini, and the comparison only covered a small number of files due to API quota.

9. The RFC corpus covers only the RFCs judged relevant to the seven fault types in scope. It covers the specification, not real-world implementation behaviour. A real router does not always follow its specification exactly. A correct citation against the RFC text can still miss how a specific vendor's implementation actually handles a case.

10. A correction method's own judge marking a file resolved does not guarantee the explanation is correct.

6.3 Future Work

Dataset scale and diversity - The current corpus covers two topologies. Both were deliberately simple starting points, not a representative sample of real-world network scale. A learned model, discussed below, needs a wider range of topology sizes and shapes to train reliably. The current corpus was not built with that scale in mind.

Extending dataset generation across a broader set of topologies is a precondition for the detection work below, closer to the size and complexity of real operator networks.

Detection at greater scale - Each new fault type or trigger mechanism requires its own hand-written rule. This approach does not scale as the range of faults in scope expands. **PCAP2STORY** currently covers three protocols, BGP, OSPF, and BFD, and seven fault types within them. Real deployments involve far more protocols and failure modes than any fixed rule set can practically enumerate. Every real capture in this corpus also came from a single vendor's routing software FRR. Whether the same rules or a learned successor, generalise to other vendors' implementations of the same protocols still needs to be tested.

Graph neural networks have already shown strong results on related network anomaly problems, detecting large-scale BGP anomalies with high accuracy [3]. A similar graph-based approach could support anomaly detection within **PCAP2STORY**, learning fault-shaped patterns across a topology relying on a hand-written rule per fault type or a fixed timing window. Classification of a flagged anomaly into a specific fault type could follow, once enough labelled data exists to support it. Whether this replaces the current rule modules or works alongside them as a fallback remains an open question.

Explanation grounded in more than RFC text - Explanation currently grounds every claim in RFC text alone. RFC text does not capture how real vendor implementations behave in practice. A router does not always follow its specification exactly. Grounding explanation in a wider knowledge base could address this. Vendor documentation and a curated set of real-world incident cases, alongside the RFC corpus already in place, would let an explanation draw on documented operational behaviour when the two diverge. A wider knowledge base would also let explanation suggest a next step grounded in how similar faults have actually been resolved in practice.

The evaluation used GPT-5 and GPT-5.6 exclusively, with only limited cross-provider comparison. Running the same evaluation methods against a broader set of models would show whether the defects found in Chapter 5 are specific to one model family or general to the explanation approach itself.

Human evaluation - This pipeline is fully automated, including every check used to evaluate it. A human reviewer would test whether the language model's output is actually reliable rather than only judged reliable by another model. This applies to both the RFC grounding and the recommended next steps. A combined human-and-model evaluation pipeline could catch errors current automated checks might miss.

Bibliography

- [1] New Relic. 2025 observability forecast. Technical report, New Relic, Inc., sep 2025. p. 10. <https://newrelic.com/sites/default/files/2025-09/new-relic-2025-observability-forecast-report.pdf>.
- [2] Uptime Institute. Annual outage analysis 2026. Technical report, Uptime Institute Intelligence, may 2026. Networking outages, p. 18. <https://datacenter.uptimeinstitute.com/rs/711-RIA-145/images/2026.AnnualOutageAnalysis.pdf>.
- [3] Kevin Hoarau, Pierre Ugo Tournoux, and Tahiry Razafindralambo. BGNN: Detection of BGP anomalies using graph neural networks. In *IEEE Symposium on Computers and Communications (ISCC)*, 2022. <https://hal.science/hal-03688089v1>.
- [4] Songtao Peng, Jiaqi Nie, Xincheng Shu, et al. A multi-view framework for BGP anomaly detection via graph attention network. *Computer Networks*, 214:109129, 2022. <https://doi.org/10.1016/j.comnet.2022.109129>.
- [5] Maria Andrea Romo-Chavero, Gustavo de los Ríos Alatorre, Jose Antonio Cantoral-Ceballos, et al. A hybrid model for BGP anomaly detection using median absolute deviation and machine learning. *IEEE Open Journal of the Communications Society*, 6, 2025. <https://doi.org/10.1109/OJCOMS.2025.3550010>.
- [6] Pengfei Li, Yujing Liu, Jinshu Su, and Bo Yu. Understanding bgp with large language models: Towards routing anomaly detection and root cause analysis. Technical report, National University of Defense Technology, SSRN, may 2025. Preprint, not peer reviewed. Sec. 6 (Related Work), Sec. 4 (Evaluation). https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5278780.

- [7] Hanqing Li, Melania Fedeli, Vinay Kolar, and Diego Klabjan. BEAR: BGP event analysis and reporting. arXiv:2506.04514, June 2025. <https://arxiv.org/abs/2506.04514>.
- [8] Kentik. Bgp route monitoring. Technical report, Kentik, Inc., 2026. <https://www.kentik.com/solutions/bgp-route-monitoring/>.
- [9] ThousandEyes. Bgp and route monitoring. Technical report, Cisco ThousandEyes, 2026. <https://www.thousandeyes.com/solutions/bgp-and-route-monitoring>.
- [10] CostBench. Thousandeyes vs kentik pricing (2026). Technical report, CostBench, 2026. <https://costbench.com/compare/kentik-vs-thousandeyes/>.
- [11] CostBench. Kentik pricing 2026: 3 plans from \$2,000 to \$24,000 per month. Technical report, CostBench, 2026. <https://costbench.com/software/network-traffic-analysis/kentik/>.
- [12] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, pages 1285–1298, 2017. <https://doi.org/10.1145/3133956.3134015>.
- [13] Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2021. <https://ieeexplore.ieee.org/document/9534113/>.
- [14] Zhuoyi Yang and Ian G. Harris. Logllama: Transformer-based log anomaly detection with llama. Technical report, arXiv, 2503.14849, mar 2025. <https://arxiv.org/abs/2503.14849>.
- [15] Wei Guan, Jian Cao, Shiyu Qian, Jianqi Gao, and Chun Ouyang. Logllm: Log-based anomaly detection using large language models. Technical report, arXiv, 2411.08561, feb 2025. <https://arxiv.org/abs/2411.08561>.
- [16] Dongjie Wang, Zhengzhang Chen, Yanjie Fu, Liu, et al. Disentangled causal graph learning for online unsupervised root cause analysis. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, 2023. Abstract, Sec. 4.2.3. <https://doi.org/10.1145/3580305.3599392>.

- [17] Luan Pham. Prism: Graph-free root cause analysis. Technical report, arXiv, 2601.21359, jan 2026. Related Work section, 9% Top-1 accuracy figure. <https://arxiv.org/abs/2601.21359>.
- [18] Jerome Henry, Swadhin Pradhan, and Miroslav Popovic. Evidence-grounded ensemble diagnosis of 802.11 packet captures: A multi-stage pipeline with deterministic reliability scoring. Technical report, arXiv, 2606.06871, jun 2026. <https://arxiv.org/abs/2606.06871>.
- [19] Zongzong Wu, Ming Zhao, Fengxiao Tang, and Nei Kato. Propllm: Propagation-aware scene reconstruction for network fault diagnosis. Technical report, arXiv, 2606.00582, jun 2026. 3.9% accuracy gain, 4.7% RC@1 gain, 50.8% hallucination reduction over strongest baseline. <https://arxiv.org/abs/2606.00582>.
- [20] Keyu Wu, Qianjin Yu, Manlin Mei, Liu, et al. Tn-autorca: Benchmark construction and agentic framework for self-improving alarm-based root cause analysis in telecommunication networks. Technical report, arXiv, 2507.18190, jul 2025. <https://arxiv.org/abs/2507.18190>.
- [21] Fengxiao Tang, Xiaonan Wang, Xun Yuan, Luo, et al. Msadm: Large language model (llm) assisted end-to-end network health management based on multi-scale semanticization. Technical report, arXiv, 2406.08305, 2024. Table II, p. 8. <https://arxiv.org/abs/2406.08305>.
- [22] Haodong Chen, Ziheng Zhang, Jiang, et al. Holmes: An evidence-grounded llm agent for auditable ddos investigation in cloud networks. Technical report, Xiamen University, arXiv 2601.14601, jan 2026. <https://arxiv.org/abs/2601.14601>.
- [23] Joseph H. R. Isaac, Harish Saradagam, and Nallamotheu Pardhasaradhi. 5g core fault detection and root cause analysis using machine learning and generative ai. Technical report, Global AI Accelerators (GAIA), Ericsson India, arXiv 2508.09152, aug 2025. <https://arxiv.org/abs/2508.09152>.
- [24] Lukasz Tulczyjew, Ihor Biruk, Murat Bilgic, Charles Abondo, and Nathanael Weill. PCAPVision: PCAP-based high-velocity and large-volume network failure detection. Technical report, B-Yond, August 2024. SIGCOMM Workshop on Networks for AI Computing (NAIC '24). <https://doi.org/10.1145/3672198.3673796>.

- [25] Lukasz Tulczyjew, Kinan Jarrah, Charles Abondo, Dina Bennett, and Nathanael Weill. LLMcap: Large language model for unsupervised PCAP failure detection. Technical report, B-Yond, June 2024. IEEE International Conference on Communications Workshops (ICC Workshops). <https://doi.org/10.1109/ICCWorkshops59551.2024.10615433>.
- [26] Hyun-Min Choi and Youngseok Lee. Fine-tuning Llama3 for integrated analysis of network packet and system log data. Technical report, Chungnam National University, November 2025. 30th Asia-Pacific Conference on Communications (APCC 2025). <https://doi.org/10.23919/APCC64555.2025.11279921>.
- [27] Mingwei Zheng, Chengpeng Wang, Liu, et al. Ai agent for auditing protocol implementations against rfc specifications. Technical report, Purdue University, arXiv 2506.00714, 2025. <https://arxiv.org/abs/2506.00714>.
- [28] Mrigank Pawagi, Lize Shao, Lee, et al. Rfcscope: Detecting logical ambiguities in internet protocol specifications. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025. <https://doi.org/10.1109/ASE63991.2025.00106>.
- [29] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. Technical report, arXiv, 2303.17651, 2023. NeurIPS 2023. <https://arxiv.org/abs/2303.17651>.
- [30] Shen Yin, David Ken, and Joel Stremmel. Denoising iterative self-correction: Structured verification loops for reliable llm reasoning. Technical report, Thomson Reuters Labs, arXiv 2606.21724, jun 2026. <https://arxiv.org/abs/2606.21724>.
- [31] Thuy-A Nguyen et al. Debating to verify: A robust and explainable multi-agent LLM system for fact-checking. *Information Processing & Management (ScienceDirect)*, may 2026. <https://www.sciencedirect.com/science/article/pii/S2405959526000883>.
- [32] Lianmin Zheng et al. Judging llm-as-a-judge with mt-bench and chatbot arena. Technical report, arXiv, 2306.05685, 2023. NeurIPS 2023 Datasets and Benchmarks Track. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

- [33] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Tan, et al. A survey on llm-as-a-judge. *The Innovation*, 7(6):101253, jan 2026. <https://www.sciencedirect.com/science/article/pii/S2666675825004564>.
- [34] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 2024. <https://www.nature.com/articles/s41586-024-07421-0>.
- [35] Qiyao Sun, Xingming Li, Xixiang He, Cheng, et al. Efficient hallucination detection: Adaptive bayesian estimation of semantic entropy with guided semantic exploration. Technical report, arXiv, 2603.22812, mar 2026. 50% fewer samples, 12.6% AUROC improvement. <https://arxiv.org/abs/2603.22812>.
- [36] Ahmed Cherif. Halluscan: A systematic benchmark for detecting and mitigating hallucinations in instruction-following llms. Technical report, Sofrecom Tunisia, Orange Innovation, arXiv 2605.02443, may 2026. Sec. 3.2.5 (NLI-based detection). <https://arxiv.org/abs/2605.02443>.
- [37] Huanxin Sheng, Xinyi Liu, Hangfeng He, Jieyu Zhao, and Jian Kang. Analyzing uncertainty of llm-as-a-judge: Interval evaluations with conformal prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11286–11328, nov 2025. <https://aclanthology.org/2025.emnlp-main.569/>.
- [38] Max Prior, Natalia Milanova, and Andreas Schultz. Chunking german legal code. Technical report, Technical University of Munich, arXiv 2605.19806, 2026. <https://arxiv.org/pdf/2605.19806>.
- [39] Mateusz Śmigielski, Michał Rajkowski, Zbrocki, et al. Chunking methods on retrieval-augmented generation – effectiveness evaluation against computational cost and limitations. Technical report, Wrocław University of Science and Technology, arXiv 2606.00881, may 2026. Sec. 4, Table 4 (chunking time vs. retrieval quality). <https://arxiv.org/abs/2606.00881>.
- [40] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. Technical report, Microsoft Research, 2020. NeurIPS 2020, pp. 16857–16867. <https://papers.nips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html>.

- [41] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. Technical report, Microsoft Research, 2020. NeurIPS 2020. <https://papers.nips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [42] OpenAI. GPT-5 system card. Technical report, OpenAI, 2025. arXiv:2601.03267. <https://cdn.openai.com/gpt-5-system-card.pdf>.
- [43] Yilun Du, Shuang Li, Antonio Torralba, et al. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763, 2024. <https://proceedings.mlr.press/v235/du24e.html>.
- [44] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. Technical report, arXiv, 1904.09675, 2020. ICLR 2020. <https://arxiv.org/abs/1904.09675>.
- [45] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020. Intrinsic hallucination defined as generated content that contradicts the source, p. 1907. <https://aclanthology.org/2020.acl-main.173/>.

Appendix A

Dataset Generation Details

Chapter 3 gives a summary of the dataset **PCAP2STORY** uses. This appendix gives the underlying detail: what each trigger mechanism actually does and how many scenarios exist for it.

A.1 Real Captures

A.1.1 Trigger Mechanisms

Each fault type below is described by the specific mechanisms used to capture it.

BFD timeout - Bidirectional Forwarding Detection works by sending a fast, lightweight heartbeat between two routers, much faster than BGP's own keepalive. The link is taken down at the interface level while BFD is active on the session. Since BFD detects the loss of that heartbeat within milliseconds, it declares the link down almost immediately and tells BGP to tear down the session right away.

Hold timer expiry - Every BGP session has a hold timer. Each side expects to hear a keepalive message from its peer at regular intervals, and if no keepalive arrives before the hold timer runs out, that side assumes its peer is gone and tears the session down itself. This mechanism disables BFD first, then takes the interface down, so there is no fast heartbeat to catch the failure early. The peer receives no keepalives for the full duration of the hold timer before it concludes the session has failed. This produces a much slower, timer-bound failure than the BFD case above.

TCP connection failure - This mechanism blocks outbound TCP traffic on the BGP port at the affected PE using a local firewall rule, so the PE's own packets are silently dropped before they ever leave the machine.

Administrative shutdown - This is not a failure at all, it is a deliberate action, the

BGP session is shut down directly through the routing daemon's own administrative command, the same way an operator would take a session down on purpose for planned maintenance. Because it is deliberate, the router sends an explicit Cease notification with an Administrative Shutdown reason code before closing the session, rather than simply going silent. This is what lets the detector tell a planned shutdown apart from an unplanned failure that looks similar on the surface.

BGP daemon kill - Rather than shutting the session down cleanly, this mechanism terminates the entire routing daemon process outright, consistent with a process crash or an out-of-memory termination. Because the process is gone, it cannot send any notification to its peers before it disappears. The peer only finds out something is wrong when its own TCP connection to that process abruptly closes, with no BGP-layer message explaining why, which is a meaningfully different wire signature from every other RR Down mechanism in this dataset.

Import-only mismatch - Every route carries a Route Target, and each PE checks incoming routes against its own configured import policy to decide whether to accept them. This mechanism changes that import policy on one PE so it filters differently from the rest of the network, while leaving that PE's own outbound advertisements completely unchanged. Because the filtering decision happens locally, after the route has already arrived, nothing about the packets travelling across the network changes at all.

Auto-derived mismatch - Normally a PE is explicitly configured with the Route Target it should advertise. This mechanism removes that explicit configuration, so the routing daemon falls back to a default value it derives automatically from its own local configuration instead. That automatically derived value does not match what the rest of the network expects. Unlike the import-only case above, this wrong value is actually carried inside the advertised route itself, so it is visible on the wire, any PE inspecting the route can see the mismatch directly.

Shared Route Distinguisher - The Route Distinguisher exists specifically to keep otherwise identical routes from different PEs distinct from one another. This mechanism configures two or more PEs with the same Route Distinguisher on purpose, so the network can no longer reliably tell which PE a given route actually came from. The two PEs' routes effectively compete for the same identity, which shows up on the wire as repeated withdrawal and re-advertisement, as the network repeatedly attempts to converge on a single, consistent route for the identifier.

Clean move - EVPN keeps track of which PE is currently responsible for reaching

a given MAC address. This mechanism manually re-associates a MAC address with a different PE than the one it was previously reachable through, the same event that happens naturally when a real device physically moves. The PE that used to own that MAC address withdraws its route for it, and the new PE advertises a route for it instead, and that withdraw-then-advertise pair is the signal the detector actually looks for.

A.1.2 Scenario Counts

Capture windows were tuned individually per mechanism, since mechanisms driven by a longer protocol timer need a longer window to reliably capture the full wire signature. Table A.1 gives the number of scenarios generated for each fault type and mechanism, on both topologies.

Fault type	Trigger mechanism	5PE/2RR	10PE/3RR
Link Down	BFD timeout	10	20
Link Down	Hold timer expiry	10	20
Link Down	TCP connection failure	10	20
PE Cease	Administrative shutdown	10	20
RR Down	BGP daemon kill	4	6
RR Down	Graceful administrative shutdown	4	6
RT Misconfiguration	Import-only mismatch	10	20
RT Misconfiguration	Auto-derived mismatch	10	20
RD Collision	Shared Route Distinguisher	7	10
MAC Mobility	Clean move	3	20

Table A.1: Real scenarios by fault type and mechanism.

A further 26 scenarios (5PE/2RR) and 27 scenarios (10PE/3RR) combine two fault types within a single capture, or test the same fault with a delayed recovery, to check how the detector handles overlapping or slow-resolving faults. Together with 18 and 58 Normal captures respectively, this gives 122 total scenarios for the 5PE/2RR topology and 247 for the 10PE/3RR topology.

A.2 Synthetic Captures

A.2.1 Generation Method

The three affected mechanisms are described below.

AC-state DF toggle - Unlike the other ESDF Toggle mechanisms, this one never withdraws the segment's route at all. The existing route is instead re-advertised with its

Designated Forwarder state field changed, the same signal a real re-election produces without any session ever going down.

Single toggle - The baseline mechanism. The shared segment's route is withdrawn, then re-advertised shortly after, representing one clean, isolated change in which PE holds the Designated Forwarder role.

Type-1 EVI variant - A Designated Forwarder change signalled through a narrower, per-EVPN-Instance route, rather than the segment-level route the other ESDF Toggle mechanisms use.

Full DF election failure - An election in which no PE ends up holding the Designated Forwarder role, rather than the role simply moving from one PE to another. Both PEs sharing the Ethernet Segment withdraw their route almost simultaneously, a two-sided withdrawal rather than one PE's isolated change.

Slow toggle - Uses the same withdraw-then-advertise construction as the single toggle case, but stretched across two full cycles instead of one, with a longer down period between the withdrawal and the re-advertisement each time.

Repeated toggle - Uses the same single toggle construction, but repeated several times in quick succession within one capture. What distinguishes this case from the baseline is how often the toggle recurs, not a different kind of change.

Single toggle, mid-churn - Uses the same single toggle construction, but the change is injected while the network already has other background activity happening, so the genuine toggle has to be told apart from surrounding noise.

No-recovery variant - Uses the same withdrawal as the single toggle case, but no re-advertisement ever follows. The segment is left permanently withdrawn for the remainder of the capture, unlike every other ESDF Toggle mechanism, which recovers.

Rapid flap - A single MAC address move from one PE to another, marked with a sequence number identifying it as one distinct move. The address is withdrawn from its old PE and re-advertised from the new one shortly after.

Repeated flap - The same kind of move as rapid flap, occurring several times within one capture. Each move's sequence number increments so the individual moves can be told apart, and ownership of the address alternates back and forth between the two PEs across the sequence.

RT Misconfiguration (ES-Import) - This mechanism represents a Route Target import policy misconfiguration scoped to a specific Ethernet Segment, rather than the PE-wide misconfiguration the two real-corpus RT Misconfiguration mechanisms represent. The generator constructs a route carrying a deliberately incorrect Route

Target community for this narrower, per-segment policy, making the mismatch visible directly in the advertised route rather than only in local import filtering. The import-only mismatch described earlier produces no such visibility, since its filtering happens after the route has already been accepted unchanged. This mechanism differs from that case in exactly this respect, the incorrect value is carried in the route itself, so it is visible on the wire rather than confined to a PE’s local import decision.

A.2.2 Scenario Counts

Table A.2 gives the scenario count for each fault type and mechanism, on both topologies.

Fault type	Trigger mechanism	5PE/2RR-matched	10PE/3RR-matched
ESDF Toggle	AC-state DF toggle	2	4
ESDF Toggle	Single toggle	2	4
ESDF Toggle	Type-1 EVI variant	2	4
ESDF Toggle	Full DF election failure	2	6
ESDF Toggle	Slow toggle	1	1
ESDF Toggle	Repeated toggle	2	4
ESDF Toggle	Single toggle, mid-churn	2	4
ESDF Toggle	No-recovery variant	2	4
MAC Mobility	Rapid flap	2	2
MAC Mobility	Repeated flap	2	2
RT Misconfiguration	ES-Import RT mismatch	4	8

Table A.2: Synthetic scenarios by fault type and mechanism.

The 10PE/3RR full DF election failure count reflects three PE-pair-specific variants, since this topology has two eligible multihomed ES pairs against the 5PE/2RR topology’s one. A further 5 scenarios in the 5PE/2RR synthetic corpus, and 3 scenarios in the 10PE/3RR synthetic corpus, each combine two fault types within a single capture. This gives 28 total synthetic scenarios for the 5PE/2RR corpus and 46 for the 10PE/3RR corpus.

A.3 Infrastructure Challenges

Building a working testbed required resolving several infrastructure problems.

FRR image instability - The default FRR image (`frrouting/frr:latest`) produced a crash loop in zebra, FRR’s core routing manager process. This made the image unusable for any sustained capture. This was resolved by pinning to a specific, stable release, `quay.io/frrouting/frr:10.6.1`.

Docker Desktop's WSL2 integration - The testbed was intended to run under native Docker CE inside WSL2, without Docker Desktop involved at all. Partway through the project, Docker Desktop's WSL2 integration setting was found re-enabled without having been changed intentionally. Whenever this setting was on, the testbed's networking broke abruptly, no session or link would establish properly, making the entire setup unusable. Significant debugging time went into tracing the failure back to this one setting, since the exact cause of it reverting was never identified. The fix was manually confirming this setting stayed disabled, and that native Docker CE remained the only runtime in use going forward.

FRR daemon watchdog instability - FRR ships with a daemon watchdog process (`watchfrr`) whose job is to detect a crashed routing daemon and restart it automatically, keeping the router self-healing without manual intervention. In this testbed, `watchfrr` hit a known upstream bug that caused it to treat healthy daemons as crashed, restarting them on their own every 30 to 90 seconds regardless of whether anything had actually failed. This was resolved with a custom startup script that bypasses `watchfrr` entirely, starting each daemon directly instead.

Shared Ethernet Segment configuration - Establishing a shared Ethernet Segment requires a bonded access interface on the connecting PEs. A bonded interface combines several physical links into one logical link. A simple point-to-point link does not satisfy this requirement. Zebra, FRR's core routing manager process, only recognises a genuine bonded interface as a valid access port for this purpose. The testbed's original access interface was a dummy interface, a placeholder link with no real bonding behind it. Because zebra's check for a valid bonded interface never matched a dummy interface, this failed silently, no error was raised, and the Ethernet Segment table simply never populated. On the five PE, two RR topology, this was resolved by replacing the dummy interface with a real bonded interface, backed by dedicated stub-node slave links, confirmed working with a populated VNI count on both connected PEs. But the resulting connection was not stable enough to capture a single complete fault scenario. On the ten PE, three RR topology, the same fix did not resolve the issue. The Ethernet Segment table still failed to populate even after upgrading FRR, indicating an additional unmet requirement beyond interface type alone that was not identified. This made shared Ethernet Segment scenarios unreliable to capture reliably across both topologies, so ESDF Toggle and the ES-Import Route Target mismatch variant were generated synthetically instead, described in Section 3.3.

Appendix B

Rule Module Details

Chapter 4 covers RT Misconfiguration and RR Down in full. This appendix covers the remaining five modules.

Link Down - RFC 4271 defines session teardown, which is precisely what a Link Down is. RR Down shares this same RFC, since it involves sessions going down, many at once instead of one. The two faults follow the same underlying pattern, differing only in breadth. A BFD state change to Down identifies a BFD timeout. The diagnostic code carried is 1, meaning Control Detection Time Expired. A hold timer expiry is identified by a TCP FIN for the same PE, preceded by a BFD state change carrying diagnostic code 3, meaning Neighbor Signaled Down. A TCP connection failure is identified by a TCP reset, or a TCP FIN with no such preceding code.

The module excludes one specific non-faulty pattern before accepting a candidate: RFC 4271's connection collision resolution procedure. When two routers briefly open two TCP connections to each other at once, one side deliberately tears down the duplicate, and this teardown can look identical to a real failure on the wire. It is excluded by two conditions together, the teardown is immediately preceded by a notification carrying a collision-tail subcode, and the session it belongs to had only just been established. The collision window itself is 0.01 seconds, set from real observed timing with a generous margin. A second, separate window of 2 seconds confirms the session was genuinely recently established before excluding it on this basis.

A withdrawal window of 7 seconds decides which route withdrawals belong to the same incident. Recovery is identified by a new session established event on the same PE. This search checks the rest of the file with no time limit, so it finds recovery whenever it genuinely happens.

PE Cease - A deliberate shutdown produces a BGP NOTIFICATION carrying Cease, Administrative Shutdown, defined in RFC 4271 and further specified by RFC 4486. This subcode alone distinguishes a genuine PE Cease from every other teardown, including the connection collision case Link Down and RR Down must otherwise exclude, so no separate exclusion logic is needed here.

Each matching notification stands as its own incident, with no grouping across multiple events. Recovery is identified by a new session established event on the same node, searched with no time limit, so recovery is found whenever it genuinely happens.

RD Collision - RFC 4364's Route Distinguisher exists to keep routes from different PEs distinct, even when the routes themselves carry the same content. The module identifies a collision structurally, two or more PEs advertising under the same Route Distinguisher, read directly from each route's content. No timing window is needed for this, since the trigger is not about events happening close together, it is simply whether the same Route Distinguisher genuinely appears on more than one PE at once. There is no single root cause node, since a collision is inherently the fault of every colliding PE together, not one PE acting alone. The output fields reflect this, shown in Listing B.1.

```
{
  "fault_type": "RD Collision",
  "trigger_mechanism": "Shared Route Distinguisher (RD Collision)",
  "affected_nodes": ["PE3", "PE4", "PE5"],
  "time_of_first_fault": 1785495684.085137,
  "time_of_first_fault_readable": "2026-07-31T11:01:24.085Z",
  "recovery_status": "NOT_RECOVERED",
  "colliding_route_distinguisher": "65000:999",
  "colliding_routes": {
    "PE3": ["Type-2 (MAC/IP Advertisement)",
           "Type-3 (Inclusive Multicast Ethernet Tag)"],
    "PE4": ["Type-2 (MAC/IP Advertisement)",
           "Type-3 (Inclusive Multicast Ethernet Tag)"],
    "PE5": ["Type-2 (MAC/IP Advertisement)",
           "Type-3 (Inclusive Multicast Ethernet Tag)"]
  }
}
```

Listing B.1: Detection output for an RD Collision incident spanning three PEs.

Recovery requires every colliding PE to individually revert to its own correct Route Distinguisher. The search for this has no time limit, so recovery is found whenever it

genuinely happens, and the incident is only marked recovered once every colliding PE has reverted, not just one.

MAC Mobility - RFC 7432 §15 defines a genuine move as a withdrawal from one PE followed by a re-advertisement of the same address from a different PE. The module checks every withdrawal event in the file, not just the first. For each one, it scans every later event. There is no time limit. It stops once it finds that withdrawal's matching readvertisement of the same MAC. There is no single root cause node for this fault, since a move inherently involves two PEs, the one losing the address and the one gaining it, so the output names both directly.

MAC identity alone is not enough to confirm a move, since the same address reappearing much later could be unrelated background activity rather than an actual transfer. The module confirms a real move by checking that the matched readvertisement is the first readvertisement of that MAC anywhere in the file after the withdrawal, and that the origin node never readvertises the same address again before that match. If the address ever reappears at its original PE first, the address never actually left, and the candidate is rejected.

A MAC reachable through two PEs at once is not automatically a move, since that is exactly what a shared Ethernet Segment is meant to allow. The module looks at the full history of advertisements and withdrawals for that MAC across the entire file. If the destination PE already advertised the same MAC before the earliest withdrawal in that history, the address was already reachable there, and the candidate is rejected as pre-existing ownership rather than a genuine move.

Once a move is confirmed, the module counts how many distinct moves of the same address occur in the file. A single move is reported as a clean move. Two or more are reported as a flap. The field reported for this fault is `move_completed`, not a recovery status, since a move is not a session going down and coming back. It is set true once both the withdrawal and the matching re-advertisement are confirmed present, with the time of re-advertisement itself reported alongside it. A sample output is shown in Listing [B.2](#).

ESDF Toggle - Designated Forwarder election is defined jointly by RFC 7432 and RFC 8584. Four sub-triggers are checked in a fixed order: a full Type-4 Ethernet Segment route withdrawal first, then a Type-1 route carrying a per-EVI tag, then a per-segment route confirmed by both PEs sharing the segment, then a direct AC-state field change. The order matters because a route matching the second check could, in principle, also carry a signal the third check would catch, checking in this order means

```
{
  "fault_type": "MAC Mobility",
  "trigger_mechanism": "clean_move",
  "affected_node_pair": { "origin": "PE3", "destination": "PE4"},
  "time_of_first_fault": 1785505842.074224,
  "time_of_first_fault_readable": "2026-07-31T13:50:42.074Z",
  "move_completed_time": 1785505843.897125,
  "move_completed_time_readable": "2026-07-31T13:50:43.897Z",
  "sequence_incremented": false,
  "move_completed": true
}
```

Listing B.2: Detection output for a clean MAC Mobility move from PE3 to PE4.

the first, more specific match wins.

Every withdrawal in the file is checked, with no time limit on the search that follows each one. As with MAC Mobility, matching by content alone is not enough. A genuine toggle requires the matched readvertisement to be the first one found for that route after the withdrawal, and the original PE must not reclaim the Designated Forwarder role before that match.

Appendix C

Explanation Examples

Section 4.4 gives one worked example, for RD Collision. This appendix gives one representative example for each of the remaining fault types the corpus covers, plus examples of a multi-incident capture, a confirmed co-occurrence, an artifact-demoted detection, a structurally undetectable case, and a defect found and fixed during evaluation.

Link Down

FILE: A Link Down on a single PE, triggered by a BGP Hold Timer expiry.

SUMMARY: Link Down on XPE1 caused by a BGP Hold Timer expiry started at 2026-08-01T12:02:15.968Z; the fault recovered at 2026-08-01T12:02:52.134Z.

NEXT STEPS: The Link Down fault self-resolved at 2026-08-01T12:02:52.134Z; no corrective action is required.

RFC CITATIONS: RFC 4271 Section 6.5 (Hold Timer Expired Error Handling); RFC 4271 Section 10 (BGP Timers); RFC 4271 Section 4.4 (KEEPALIVE Message Format)

RFC GROUNDING: RFC 4271 Section 6.5 (Hold Timer Expired Error Handling) specifies that if successive KEEPALIVE, UPDATE, and/or NOTIFICATION messages are not received within the negotiated Hold Time, the peer sends a NOTIFICATION with the Hold Timer Expired error and closes the BGP connection. The observed "Hold Timer Expired" trigger and resulting Link Down on XPE1 are exactly the mandated behavior for this condition. RFC 4271 Section 4.4 (KEEPALIVE Message Format) requires KEEPALIVE messages be exchanged often enough to prevent Hold Time expiry (with a reasonable maximum interval of one third of the Hold Time). The link failure aligns with a lapse in KEEPALIVE (or other qualifying) messages long enough for the Hold Timer to expire. RFC 4271 Section 10 (BGP Timers) defines the HoldTimer and KeepaliveTimer, their default relationship (Keepalive at approximately one third of HoldTime), and that the HoldTimer is per-peer

configurable. This validates that a timer-driven teardown is expected protocol behavior when the HoldTimer reaches zero without receiving qualifying messages, matching the fault observed on XPE1.

RR Down

FILE: An RR Down on a route reflector with four attached PE clients, triggered by a graceful administrative shutdown.

SUMMARY: RR Down on XRR1 due to Cease/Administrative Shutdown began at 2026-08-01T17:30:12.755Z and the fault recovered at 2026-08-01T17:30:27.354Z. A concurrent Link Down on XPE1 (TcpConnectionFails) began at 2026-08-01T17:30:12.755Z and recovered at 2026-08-01T17:30:26.174Z; per the provided causal note, this was the same underlying event as the XRR1 RR Down (not an independent fault), detected on the client 0.000309s after the RR event and within the stated 1.060s, 4-client fan-out window.

NEXT STEPS: The RR Down fault self-resolved at 2026-08-01T17:30:27.354Z; no corrective action is required. The Link Down fault self-resolved at 2026-08-01T17:30:26.174Z; no corrective action is required.

RFC CITATIONS: RFC 4724 Section 4.2 (Procedures for the Receiving Speaker); RFC 8538 Section 4.1 (Rules for the Receiving Speaker); RFC 4271 Section 8.1.4 (TCP Connection-Based Events)

RFC GROUNDING: RFC 4271 Section 8.1.4 defines Event 18 (TcpConnectionFails) as a mandatory BGP FSM event indicating a TCP connection failure, including cases where the remote peer sends a FIN or a local timeout downs the connection. This maps directly to the XPE1 detection of Link Down with trigger mechanism TcpConnectionFails at 2026-08-01T17:30:12.755Z, consistent with a control-plane session drop to its route reflector. RFC 4724 Section 4.2 specifies the receiving speaker behavior when a peer restarts and Graceful Restart is in use: upon detecting TCP session termination, retain previously learned routes as stale, and upon re-establishment, replace stale routes and remove any remaining stale entries after End-of-RIB. These rules are the applicable standards context for the RR Down on XRR1 caused by an administrative shutdown; the facts only establish the shutdown and recovery by 2026-08-01T17:30:27.354Z, without establishing that a restart or GR-based re-establishment occurred. RFC 8538 Section 4.1 updates the RFC 4724 receiving-speaker rules by making the stale timer mandatory and clarifying consecutive-restart handling. However, the detector facts do not establish that XRR1 advertised the Graceful Restart Capability to its clients; therefore, we do not assert that stale-route retention

or a mandatory stale timer applied during this incident. Relationship between incidents: the causal note states the Link Down on XPE1 is a confirmed co-occurrence of the same event as the XRR1 RR Down (not a separate failure), observed 0.000309s after the RR trigger and within the 1.060s fan-out window for XRR1's 4 PE clients.

PE Cease

FILE: A PE Cease on a single PE, not yet recovered by the end of the capture.

SUMMARY: A BGP Cease (Administrative Shutdown) was observed on PE1 at 2026-07-31T17:31:04.276Z, and the fault had not recovered by the end of the capture. The concurrent Link Down detection (TcpConnectionFails) on PE1 is the same underlying event, not an independent fault.

NEXT STEPS: NEXT STEP: Verify whether an administrative shutdown was applied on PE1 and, if unintended, remove it (no shutdown) on the BGP/EVPN session to its RR to restore reachability. UNCERTAIN -- the capture proves a Cease with Administrative Shutdown and the resulting TCP failure on PE1, but operator intent (planned maintenance vs. error) is unknown.

RFC CITATIONS: RFC 4486 Section 3 (Subcode Definition); RFC 8538 Section 5.1 (When to Send a Hard Reset); RFC 4271 Section 8.1.4 (TCP Connection-Based Events); RFC 4271 Section 8.1.5 (BGP Message-Based Events)

RFC GROUNDING: RFC 4486 Section 3 (Subcode Definition) enumerates Cease NOTIFICATION subcodes and defines "Administrative Shutdown" (subcode 2). The capture's Cease/Administrative Shutdown trigger on PE1 matches this definition, classifying the event as a Cease with subcode 2 without asserting operator intent. RFC 8538 Section 5.1 (When to Send a Hard Reset) advises using a Hard Reset for Cease subcodes including Administrative Shutdown, but only as a suggestion, not a requirement; the incident facts do not establish that a Hard Reset was used, so this RFC alone does not prove the teardown mechanism in this case. RFC 4271 Section 8.1.4 (TCP Connection-Based Events) defines Event 18: TcpConnectionFails as a mandatory event when the TCP connection fails (e.g., FIN exchange or timeout). The Link Down detection labeled TcpConnectionFails on PE1 is consistent with the expected TCP outcome when a peer sends/acts on a Cease (Administrative Shutdown). RFC 4271 Section 8.1.5 (BGP Message-Based Events) defines Event 25: NotifMsg for reception of a NOTIFICATION (such as Cease). The excerpt only defines receipt of the NOTIFICATION; it does not state that this event by itself tears down the session or directly triggers TcpConnectionFails. Taken together,

these clauses do not themselves establish that a Cease NOTIFICATION causes a TCP failure; they define the NOTIFICATION and the TCP failure as separate events. The provided co-detection note and identical start time on PE1 confirm the Link Down and Cease reports are the same underlying event rather than separate faults.

RT Misconfiguration

FILE: An RT Misconfiguration on a single PE, triggered by an auto-derived mismatch, not yet recovered.

SUMMARY: An RT Misconfiguration on PE1 (triggered by Auto-Derived Mismatch) began at 2026-07-31T17:44:39.143Z; the fault had not recovered by the end of the capture. PE1's Route Target did not match the expected 100:1; no observed route-reflection, route-import, or connectivity failure is evidenced in the supplied facts.

NEXT STEPS: NEXT STEP: Verify and correct PE1's Route Target configuration to match the expected 100:1 for the affected service (import and/or export as applicable). UNCERTAIN -- The detector facts only indicate an auto-derived RT mismatch on PE1 and do not specify whether the import RT, export RT, or both differ.

RFC CITATIONS: RFC 4364 Section 4.3.5 (Building VPNs Using Route Targets); RFC 4364 Section 12 (Management VPNs)

RFC GROUNDING: RFC 4364 Section 4.3.5 states that correct assignment of Import and Export Route Targets is how membership and reachability within a VPN (e.g., a fully meshed closed user group) are achieved. If a PE uses an unexpected RT, its routes will not be imported by the intended peers, or it will fail to import the intended routes, disrupting VPN connectivity. RFC 4364 Section 12 provides an explicit example that correct use of specific RTs (e.g., T1/T2) enforces the intended communication patterns; this underscores that RT values are the control mechanism for distributing routes to the right VRFs. Applied here: all PEs are expected to use RT 100:1; PE1's RT misconfiguration means its BGP updates may not be distributed/imported as intended through the route-reflector topology, risking loss of VPN membership for PE1's routes until the RT is corrected, though the supplied facts do not establish any observed distribution/import or connectivity failure.

MAC Mobility

FILE: A clean MAC Mobility move from PE3 to PE4.

SUMMARY: A MAC address moved from PE3 to PE4 starting at 2026-07-31T13:50:42.074Z, with the destination advertisement confirmed at 2026-07-31T13:50:43.897Z. This is a single, completed move, not a flap.

NEXT STEPS: The move completed successfully with no repeated flapping observed; no corrective action is required.

RFC CITATIONS: RFC 7432 Section 15 (MAC Mobility)

RFC GROUNDING: RFC 7432 Section 15 defines a genuine MAC move as a withdrawal of the address from its original PE followed by a re-advertisement from a different PE, carrying a MAC Mobility Extended Community with an incremented sequence number to distinguish it from a stale or duplicate advertisement. The withdrawal from PE3 followed by PE4's advertisement of the same address 1.823 seconds later matches this definition directly. The sequence number was not incremented, consistent with this being the address's first recorded move in the capture rather than a repeated flap between the two PEs.

Multi-Incident Detection

FILE: A capture combining a PE Cease/Link Down pair on one node with an unrelated RD Collision on two other nodes.

SUMMARY: XPE2 experienced a BGP session failure (Link Down triggered by TcpConnectionFails) concurrent with a Cease/Administrative Shutdown, beginning at 2026-08-03T14:16:10.596Z and recovering at 2026-08-03T14:16:20.945Z; these two symptoms co-occurred on XPE2 with identical timestamps, but the facts do not establish that they are a single event. Separately, XPE8 and XPE9 have a Shared Route Distinguisher (RD Collision) that began at 2026-08-03T14:16:29.030Z and had not recovered by the end of the capture; it is unrelated to the XPE2 issue.

NEXT STEPS: NEXT STEP: Reconfigure XPE8 and XPE9 so each MAC-VRF uses a unique per-PE RD matching the expected values (XPE8 10.0.0.18:2 and XPE9 10.0.0.19:2), then clear the EVPN address-family on both to republish with corrected RDs. CERTAIN -- The capture identifies a persistent Shared Route Distinguisher (RD Collision) between XPE8 and XPE9, and RFC 7432 Section 7.9 recommends per-PE-unique Type 1 RDs based on the PE's IP to avoid such collisions.

RFC CITATIONS: RFC 4271 Section 8.1.4 (TCP Connection-Based Events); RFC 4271 Section 8.1.5 (BGP Message-Based Events); RFC 8538 Section 5.1 (When to Send a Hard Reset); RFC 4486 Section 3 (Subcode Definition); RFC 7432

Section 7.9 (Route Distinguisher Assignment per MAC-VRF); RFC 4364 Section 4.3.6 (Route Distribution Among VRFs in a Single PE)

RFC GROUNDING: RFC 4271 Section 8.1.4 defines `TcpConnectionFails` (Event 18) as a mandatory event indicating a TCP connection failure (e.g., FIN exchange or timeout). The Link Down on XPE2 was explicitly triggered by `TcpConnectionFails`, which maps directly to a BGP transport failure and explains the brief session drop and recovery between 2026-08-03T14:16:10.596Z and 2026-08-03T14:16:20.945Z. RFC 4271 Section 8.1.5 enumerates message-based events (e.g., OPEN, KEEPALIVE, UPDATE) and explicitly notes that `OpenCollisionDump` can occur in Idle, Connect, Active, OpenSent, and OpenConfirm as well as Established; after a `TcpConnectionFails` event, the session leaves the established state, which is consistent with the observed transient outage on XPE2. RFC 4486 Section 3 defines the Cease NOTIFICATION subcodes, including "Administrative Shutdown" (subcode 2). The concurrent "PE Cease/Administrative Shutdown" on XPE2 precisely matches this definition, explaining why the session was intentionally torn down. RFC 8538 Section 5.1 advises using a Hard Reset for Cease subcodes such as Administrative Shutdown. This guidance is a suggestion, not a requirement; an Administrative Shutdown may use a Hard Reset leading to immediate termination, which is consistent with the observed brief outage window on XPE2. RFC 7432 Section 7.9 mandates that each MAC-VRF have an RD and that it be unique on a PE, and further recommends using a Type 1 RD whose value includes the PE's IP address plus a PE-unique number. The reported Shared RD between XPE8 and XPE9 indicates they did not follow the recommended per-PE unique assignment (expected to be 10.0.0.18:2 and 10.0.0.19:2), causing an RD collision that persisted. RFC 4364 Section 4.3.6 explains that route distribution/import between VRFs is controlled by Route Targets; with the common RT 100:1 across these PEs, EVPN routes from multiple PEs are imported into the same VPN context.

Confirmed Co-occurrence

FILE: An RR Down on a three-client route reflector, co-detected with a Link Down on one client.

SUMMARY: A Route Reflector Down event on XRR3, triggered by Cease/Administrative Shutdown, began at 2026-08-01T17:33:44.631Z and the fault recovered at 2026-08-01T17:33:57.350Z. The same underlying event was co-detected as a Link Down/`TcpConnectionFails` on client XPE8 (which is attached to XRR3) at 2026-08-01T17:33:44.631Z, occurring 0.000294s after the RR event within XRR3's fan-out window, and the fault recovered at

2026-08-01T17:33:56.401Z.

NEXT STEPS: The RR Down fault self-resolved at 2026-08-01T17:33:57.350Z; no corrective action is required. The Link Down fault self-resolved at 2026-08-01T17:33:56.401Z; no corrective action is required.

RFC CITATIONS: RFC 4271 Section 8.1.4 (TCP Connection-Based Events); RFC 4271 Section 8.1.5 (BGP Message-Based Events); RFC 4724 Section 4.2 (Procedures for the Receiving Speaker); RFC 8538 Section 4.1 (Rules for the Receiving Speaker)

RFC GROUNDING: The RR Down on XRR3 with a Cease/Administrative Shutdown is consistent with a BGP NOTIFICATION-driven teardown; RFC 4271 Section 8.1.5 defines NotifMsg (Event 25) as the FSM event generated when a NOTIFICATION message with any error other than "version error" is received. The client-side observation on XPE8 of TcpConnectionFails maps directly to RFC 4271 Section 8.1.4 Event 18, which defines a TCP connection failure notice (e.g., FIN exchange or timeout); this explains the Link Down/TcpConnectionFails detection on XPE8 toward XRR3 immediately after the RR event. These two detections are therefore the same root cause observed from different roles: XRR3's administrative shutdown and the resulting TCP/BGP session failure on its client XPE8, which is attached to XRR3. During the brief outage and restart window, RFC 4724 Section 4.2 specifies the Receiving Speaker behavior for Graceful Restart: upon detecting termination with a peer that has advertised GR, the receiver retains routes as stale and replaces them after the session is re-established (with End-of-RIB signaling completing the refresh), which aligns with a short-lived, self-recovering control-plane interruption. RFC 8538 Section 4.1 further refines those rules by making the stale timer mandatory and clarifying when previously stale routes are deleted (dependent on the "N" bit); while support for a stale timer is required, implementations may allow it to be disabled, so it does not guarantee a finite bound in all configurations.

Precedence record: CONFIRMED_COOCCURRENCE, reason: "co-occurring Link Down incident (root_cause_node=XPE8) 0.000294s later, within this RR's fan-out-scaled window (0.740s) -- same real event, not an independent fault."

Artifact-Demoted Detection

FILE: A Link Down on a single PE, triggered by BFD Down, later followed by a PE Cease detection on the same node.

SUMMARY: Link Down on PE1 triggered by BFD Down began at 2026-07-31T18:13:07.759Z; the fault recovered at 2026-07-31T18:13:39.360Z.

NEXT STEPS: The Link Down fault self-resolved at 2026-07-31T18:13:39.360Z; no corrective action is required.

RFC CITATIONS: RFC 5880 Section 6.8.16 (Administrative Control); RFC 5880 Section 6.8.18 (Holding Down Sessions)

RFC GROUNDING: RFC 5880 Section 6.8.16 (Administrative Control) states that when signaling indicates the underlying path has failed, an implementation may administratively disable the BFD session using a Path Down diagnostic, transitioning the session state to AdminDown. The supplied facts' "BFD Down" on PE1 and resulting Link Down align with this rule: the session state is not shown transitioning to AdminDown in the supplied facts; only a BFD Down-triggered Link Down on PE1 is observed, which later recovered. RFC 5880 Section 6.8.18 (Holding Down Sessions) specifies that systems may hold a BFD session in Down/AdminDown and must maintain session state (including timing) until a Detection Time passes without BFD Control packets, after which normal transmission can resume. The brief outage window from 2026-07-31T18:13:07.759Z to 2026-07-31T18:13:39.360Z reflects a transient BFD Down condition; the provided facts do not indicate whether the session was explicitly held Down/AdminDown or whether any change in BFD Control signaling timing drove the recovery.

Precedence record: a later PE Cease detection on the same node was labelled LIKELY_ARTIFACT_OF_LINK_DOWN, reason: "gap to nearest same-PE Link Down incident (31.601028s) exceeds 1.0s -- reconnect-phase artifact of an earlier, unrelated fault."

MAC Mobility + ESDF Toggle

FILE: A MAC Mobility clean move on the shared segment between XPE10 and XPE3, co-detected with an ESDF Toggle on the multihomed pair XPE3/XPE4.

SUMMARY: A MAC Mobility clean move involving XPE10 and XPE3 started at 2026-08-01T18:42:22.257Z and the fault had not recovered by the end of the capture. An ESDF Toggle on XPE3 (which is multihomed with XPE4 on ESI 03:02:00:00:34:34:00:00:22) began at 2026-08-01T18:42:22.540Z and recovered at 2026-08-01T18:42:23.810Z; per the capture, these are causally related expressions of the same underlying event on XPE3 (the ESDF Toggle followed the MAC Mobility trigger by 0.283s), not independent faults.

NEXT STEPS: NEXT STEP: Investigate on XPE3 why a Type-4 ES route was withdrawn and confirm that MAC/IP Advertisement withdrawals/updates between XPE10 and XPE3 followed the expected sequence and DF re-election occurred cleanly; check AC state toward the multihomed segment with XPE4 around 2026-08-01T18:42:22.540Z. UNCERTAIN -- the capture proves a clean

MAC move with a downstream DF re-election; the DF re-election recovered quickly while the MAC Mobility had not recovered by the end of the capture, but it does not reveal whether the ES route withdrawal was due to an expected host move or an access/link flap.

RFC CITATIONS: RFC 7432 Section 15 (MAC Mobility); RFC 8584 Section 4 (The AC-Influenced DF Election Capability); RFC 8584 Section 1 (Introduction)

RFC GROUNDING: RFC 7432 Section 15 (MAC Mobility) defines a MAC move as the MAC being advertised from two different Ethernet segments and requires that the advertising PE(s) for the previous segment withdraw their MAC/IP Advertisement routes; it also relies on the MAC Mobility extended community sequence to converge to the current location. The capture's MAC Mobility clean move between XPE10 and XPE3 fits this definition, but the facts do not provide sequence numbers or explicit withdrawals/updates sufficient to prove that the sequence-handling rules were followed. RFC 8584 Section 4 (The AC-Influenced DF Election Capability) states that the DF candidate list is built from ES routes' Originating Router's IPs and is pruned when Ethernet A-D routes are withdrawn, and that updates/withdrawals of these routes trigger DF re-election. The observed ESDF Toggle on XPE3 was triggered by a Type-4 ES route withdrawal; in this section, DF re-election is explicitly triggered by updates/withdrawals of Ethernet A-D per ES and per EVI routes that prune the candidate list. RFC 8584 Section 1 (Introduction) clarifies the DF's role for BUM traffic on a multihomed ES and formalizes the DF election FSM. Given XPE3's multihoming with XPE4, a DF change on that ES directly affects BUM forwarding responsibilities; the detected DF toggle therefore reflects a control-plane re-election event consistent with the framework described here and explains its tight coupling to the preceding MAC mobility event.

Precedence record: CONFIRMED_COOCCURRENCE, reason: "follows MAC Mobility incident 0's trigger by 0.283530s on the same (origin) PE -- genuine downstream reaction, not an independent fault."

RR Down + PE Cease

FILE: An RR Down on RR1 (BGP daemon kill), recovered, with a later PE Cease detection on the same RR demoted as an artifact of the recovery.

SUMMARY: An RR Down fault on RR1 (triggered by TCP_connection_closed) began at 2026-07-31T17:23:23.756Z and the fault recovered at 2026-07-31T17:23:44.488Z. The same underlying event caused PE3 (which peers to RR1) to report a Link Down via TcpConnectionFails at 2026-07-31T17:23:23.756Z, and it recovered at 2026-07-31T17:23:44.485Z.

NEXT STEPS: The RR Down fault self-resolved at 2026-07-31T17:23:44.488Z; no corrective action is required. The Link Down fault self-resolved at 2026-07-31T17:23:44.485Z; no corrective action is required.

RFC CITATIONS: RFC 5880 Section 6.8.18 (Holding Down Sessions); RFC 5880 Section 6.8.15 (Forwarding Plane Reset); RFC 4271 Section 8.1.4 (TCP Connection-Based Events); RFC 4271 Section 8.1.5 (BGP Message-Based Events)

RFC GROUNDING: RFC 5880 Section 6.8.18 (Holding Down Sessions) specifies that a system may hold a BFD session in Down/AdminDown and that the remote must honor the resulting receive-interval semantics until a Detection Time passes. The RR Down detection on RR1 is consistent with a BFD-level session being placed or remaining in Down state during the event, which correctly signals loss of adjacency/reachability to its clients. RFC 5880 Section 6.8.15 (Forwarding Plane Reset) mandates that if the local forwarding plane is reset, bfd.LocalDiag be set to Forwarding Plane Reset and the session state set to Down. In this incident, the capture attributes the RR1 event to a closed TCP connection; no forwarding-plane reset is indicated, so Section 6.8.15 is not implicated here. RFC 4271 Section 8.1.4 (TCP Connection-Based Events) defines Event 18: TcpConnectionFails (e.g., FIN exchange or timeout) as a mandatory state-machine event. PE3's Link Down report with trigger TcpConnectionFails at 2026-07-31T17:23:23.756Z matches this definition, indicating the BGP transport to RR1 failed during the same window as the RR1-down event. RFC 4271 Section 8.1.5 (BGP Message-Based Events) enumerates message events (e.g., BGPOpen, KeepAlive, Update) and their FSM handling. The PE3 Link Down corresponds to the TCP-layer Event 18 (Section 8.1.4); Section 8.1.5 is cited to show the distinct set of message-based events defined by BGP, separate from the transport-failure event observed here.

A separate PE Cease detection on RR1, 1.005822 seconds after this RR Down's own recovery, was demoted before reaching

Explanation:

Precedence record: LIKELY_ARTIFACT_OF_RR_DOWN_RECOVERY,
reason: "1.005822s after RR Down incident's own recovered_time
(root_cause_node=RR1), within 2.0s -- reconnect-churn artifact of the
RR's own recovery, not an independent fault."

Structurally Undetectable Case

FILE: An import-only Route Target mismatch, the mechanism described in Section 3.4 as producing no wire signal by design.

OUTPUT: No incidents were detected in this capture. All seven fault detection modules were checked against this capture's fused event timeline, and none found a matching fault pattern, consistent with this being a normal traffic baseline.

A Defect Found and Fixed: RFC 8584 Access-Circuit Mischaracterization

FILE: A dual-PE ESDF Toggle full-failure incident, the case discussed in Section 5.3.2.

Corrected output:

SUMMARY: An ESDF Toggle caused by dual-PE Type-1 Ethernet A-D per-ES withdrawals affected PE3 and PE4 on ESI 00:11:22:33:44:55:66:77:88:02 starting at 2024-03-11T23:38:19.727Z; the fault had not recovered by the end of the capture.

NEXT STEPS: NEXT STEP: Investigate and restore PE-to-CE connectivity for PE3 and PE4 on the shared Ethernet segment, then verify that both PEs advertise the expected Ethernet A-D per-ES routes. CERTAIN -- the incident identifies dual-PE withdrawal as an Ethernet-segment full failure, and the fault remained unrecovered.

RFC CITATIONS: RFC 7432 Section 17.3 (PE-to-CE Network Failures)

RFC GROUNDING: RFC 7432 Section 17.3 requires a PE to withdraw its Ethernet A-D per-ES routes when its connectivity to the multihomed CE fails, enabling remote PEs to remove that PE's next hop. The withdrawals by both PE3 and PE4 for their shared ESI therefore match the reported Ethernet-segment full failure; the section also states that remote PEs should treat MAC/IP Advertisement routes learned from the same ESI and advertising PE as withdrawn to accelerate convergence.

Before the retrieval query fix described in Section 5.3.2, this same incident was explained by citing RFC 8584 Section 1.3.2, a section covering individual access-circuit failures that does not apply to a dual-PE scenario.

RD Collision: Self-Refine Caught One File, Missed the Other

FILE A (Self-Refine corrected it): RD Collision between PE3 and PE4.

Self-Refine original (verdict_sequence=[CORRECTED, APPROVED]):

SUMMARY: RD Collision caused by a shared Route Distinguisher between PE3 and PE4 began at 2026-07-31T11:01:24.085Z and had not recovered by the end of the capture. Expected RDs are PE3=10.0.0.13:2 and PE4=10.0.0.14:2, indicating at least one PE is advertising with an incorrect or non-unique RD.

NEXT STEPS: NEXT STEP: Reconfigure unique per-PE MAC-VRF RDs on PE3 and PE4 to match the expected values (PE3=10.0.0.13:2, PE4=10.0.0.14:2) and trigger an EVPN address-family route refresh from PE3 toward RR1 and from PE4 toward RR2. CERTAIN -- The capture identifies a shared RD between PE3 and PE4, and the expected per-PE RDs are provided; correcting the RD values and refreshing EVPN routes is the definitive fix for an RD collision.

RFC CITATIONS: RFC 7432 Section 7.9 (Route Distinguisher Assignment per MAC-VRF); RFC 4364 Section 4.3.6 (Route Distribution Among VRFs in a Single PE)

RFC GROUNDING: RFC 7432 Section 7.9 requires that each MAC-VRF have an RD, and that the RD be unique across MAC-VRFs on a PE; it also recommends constructing the RD from the PE's IP address plus a PE-unique number. The expected RDs 10.0.0.13:2 (PE3) and 10.0.0.14:2 (PE4) follow this recommendation, ensuring per-PE identity in EVPN route advertisements. The observed shared RD between PE3 and PE4 contravenes this guidance, causing their EVPN routes to present the same RD and thus collide. RFC 4364 Section 4.3.6 explains that route distribution between VRFs is driven by Route Targets. In this topology, all PEs use RT 100:1, so their EVPN routes are eligible for import by the same MAC-VRFs.

FILE B (Self-Refine did not correct it): RD Collision between XPE3 and XPE4.

Self-Refine original (verdict_sequence=[APPROVED]):

SUMMARY: RD Collision on XPE3 and XPE4: both advertised EVPN routes with a shared Route Distinguisher instead of their expected per-PE RDs (XPE3 expected_rd=10.0.0.13:2, XPE4 expected_rd=10.0.0.14:2) into RT 100:1; the fault started at 2026-08-01T18:25:48.549Z and the fault recovered at 2026-08-01T18:26:04.524Z.

NEXT STEPS: The RD Collision fault self-resolved at 2026-08-01T18:26:04.524Z; no corrective action is required.

RFC CITATIONS: RFC 7432 Section 7.9 (Route Distinguisher Assignment per MAC-VRF); RFC 4364 Section 4.3.6 (Route Distribution Among VRFs in a

Single PE)

RFC GROUNDING: RFC 7432 Section 7.9 states the RD in EVPN NLRI must be the RD of the MAC-VRF that is advertising the route, and that each PE assigns an RD that is unique on that PE (commonly derived from the PE's loopback IP plus a per-PE number). In this incident, XPE3 and XPE4 used a shared RD rather than their distinct, per-PE RDs (expected 10.0.0.13:2 and 10.0.0.14:2), so EVPN NLRIs from two different advertising MAC-VRFs were not uniquely identified, resulting in an RD collision. RFC 4364 Section 4.3.6 clarifies that route distribution into other VRFs depends on Route Targets. Here, both PEs advertised into RT 100:1, so importing PEs would bring both advertisements into the same MAC-VRF context; with a shared RD, those routes lost per-origin uniqueness within that imported set, which is consistent with the observed RD collision behavior.

DISC-corrected versions of both files converge on the same accurate grounding, discussed in Section [5.3.6](#).